



Institut za matematiku i informatiku

Prirodno-matematički fakultet

Univerzitet u Kragujevcu

Skripta iz predmeta

Operativni sistemi 2

Skripta sa ispitnim pitanjima iz predmeta OS2

Upravljanje ulazno/izlaznim uređajima i raspoređivanje diska. Fajl sistemi. Jednoprocesorsko raspoređivanje (scheduling). Višeprocorsorsko raspoređivanje i raspodivljanje u realnom vremenu. Distribuirani sistemi, klijent/serveri i klasteri. Virtuelizacija.

Računarsvto u oblaku (*Cloud Computing*). Sigurnost i bezbednost računarskih sistema.

Contents

1. Arhitektura U/I Sistema.....	4
Slojevita arhitektura U/I funkcije	4
Lokalni periferijski uređaji - jednostavno	4
Fajl sistem – složeniji.....	4
2. Baferovanje.....	5
Jednostruki bafer.....	6
Dvostruki bafer	6
Kružni bafer.....	6
3. Optimizacija performansi diska	7
4. Politike raspoređivanja diska – FIFO, PRI, LIFO, SSTF	7
5. Politike raspoređivanja diska – SCAN, C-SCAN, N-SCAN, F-SCAN.....	7
6. RAID 0,1, Redundant Array of Independent Disks.....	7
7. RAID 2, 3	8
8. RAID 4, 5, 6	8
9. Keš diska	8
Zamena zasnovana na učestanosti	8
10. U/I sistema UNIX	9
11. Linus Elevator, Raspoređivač po roku	9
12. Arhitektura fajl sistema	9
13. Organizacija fajlova	10
14. Organizacija fajlova – hrpa, sekvencijalni fajl	10
15. Organizacija fajlova – indeksirani sekvencijalni fajl, indeksni fajl, direktni (heširani) fajl	11
16. Direktorijumi.....	11
17. Blokovanje fajlova.....	12
18. Modeli dodeljivanja fajlova	13
19. Upravljanje slobodnim prostorom na disku	13
20. UNIX / LINUX fajl sistem	13
21. VSFS (Very simple file system)	14
22. FFS (Fast file system)	15
23. FSCK i vođenje dnevnika	15
24. SSD – hardverska arhitektura.....	16
25. SSD – metode FTL mapiranja	17
26. SSD – skupljanje otpada, blok i hibridno mapiranje	17
27. Vrste procesorskog raspoređivanja (dugoročno, srednjeročno, kratkoročno, U/I), skup relevantnih kriterijuma.....	18
28. FCFS, RR	19
29. VRR, SPN	19
30. SRT, HRRN, FeedBack raspoređivanje	19
31. Fairshare raspoređivanje	20
32. UNIX raspoređivanje (dodaj).....	20
33. Multiprocesorsko raspoređivanje - statičko i dinamičko, multiprogramiranje pojedinačnih procesora	20

34. Raspoređivanje niti - deljenje opterećenja, grupno raspoređivanje, namenska dodela procesora, dinamičko raspoređivanje.....	20
35. Raspoređivanje u realnom vremenu, klasifikacija zadataka, osobine real-time sistema, statičko i dinamičko raspoređivanje	21
36. Periodični zadaci - Raspoređivanje sa rokovima završetaka	22
37. Aperiodični zadaci - Raspoređivanje sa rokovima zapocinjanja	22
38. Periodični zadaci - Rate Monothonic Scheduling (RMS)	23
39. Inverzija prioriteta	24
40. Klijent-server aplikacije, ispravno i neispravno korišćenje arhitekture, klase klijent/server aplikacija, troslojna arhitektura, konzistentnost keša	24
41. RPC (Remote Procedure Call), klasteri	25
42. i 43. Virtuelizacija - osnovni koncepti, prednosti i mane, kategorije, hipervozor	26
44. Cloud computing - osnovni koncepti i kategorizacije	27
45. Docker kontejneri - osnovni principi. Copy on Write, poređenje sa VM	27
46. Vrste pretnji računarskim sistemima.....	28
47. Vrste i karakteristike malware-a	28
48. Zaštita - kontrola pristupa orijentisana na korisnike, podatke.....	28
49. Otkrivanje napada, primer buffer overrun	29
50. Praktični primeri mrežnih napada (link)	29
51. Migracija procesa	30
52. Distribuirana globalna stanja - konzistentno i nekonzistentno stanje.....	30
53. Algoritam distribuiranog trenutnog snimka	31
54. Distribuirano međusobno isključenje.....	31
55. Algoritam vremenskog žiga.....	32
56. Lamportov algoritam distribuiranog reda čekanja.....	32
57. Uzajamno isključenje pomoću prosleđivanja tokena	34
58. Zookeeper - koncept Znode-a (proveri)	34
59. Zookeeper - algoritam izglasavanja leader-a (1) (proveri).....	35
60. Zookeeper - algoritam izglasavanja leader-a (2) (proveri).....	36
61. Network File System (NFS)	37

Ako primetis bilo kakvih nejasnoca pogledaj skeniranu svesku ili predavanja od Milosa.

1. Arhitektura U/I Sistema

Programirani UI – CPU izdaje komande U/I modulu, a process čeka dok se operacije ne završi

U/I modul koji se inicira prekidom – CPU izdaje U/I komandu, nastavlja da izvršava naredne instrukcije, a U/I modul ga prekida kada završi zadatu operaciju

Direktan pristup memoriji (DMA) – DMA modul upravlja razmenom podataka između U/I i glavne memorije. CPU šalje zahtev za prenos, DMA obavi prenos i prekida CPU kada završi.

Kao ulaz prihvata zahtev od CPU, zahtev za pisanje i čitanje i adrese i podatke na adresama. Izlaz je interrupt kada obradi podatke.

Ključne tačke u istorijskom razvoju: dodavanje kontrolera, dodavanje interrupta, uvođenje DMA modula, U/O modul dobija memoriju.

Ciljevi U/I modula: efikasnost i opštost (npr. da se deo modula za tastaturu iskoristi za deo modula za miš).

Slojevita arhitektura U/I funkcije

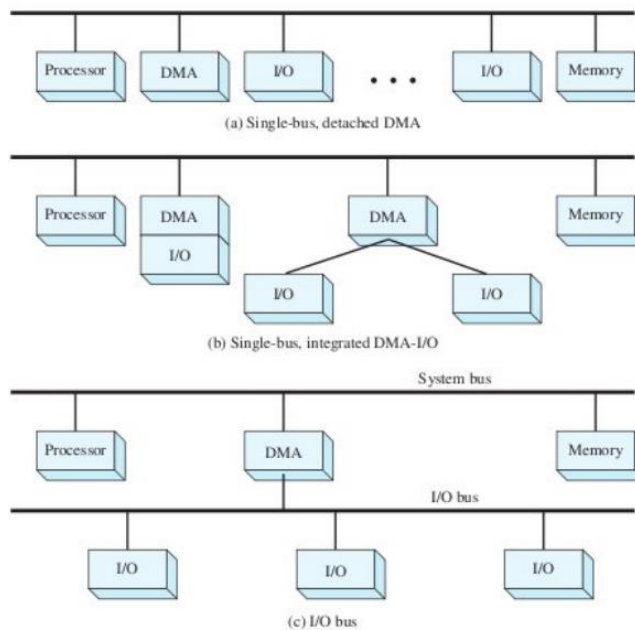
Svaki sloj izvodi podskup f-ja, tj. Višem sloju obezbeđuje odgovarajući interfejs. Daje nam mogućnost da tehnologiju jednog sloja izmenimo a da pritom interfejs prema gornjem sloju ostane nepromenjen.

Lokalni periferni uređaji - jednostavno

1. **Logički U/I** – Dozvoljava rad sa uređajem preko hardverskog ID-a, jednostavne komande
2. **U/I uređaja** – Zadane operacije se pretvaraju u sekvence U/I instrukcija, komandi kontrolera.
3. **Raspoređivanje i upravljanje** – Stvarno čekanje u redovima i raspoređivanje hardverskog nivoa.

Fajl sistem – složeniji

1. **Directory management** – Simbolička imena fajlova se pretvaraju u identifikatore
2. **Fajl sistem** – otvaranje/zatvaranje, čitanje/upis, prava pristupa
3. **Fizička organizacija** – logičke reference fajlova koje se pretvaraju u fizičke adrese diska



2. Baferovanje

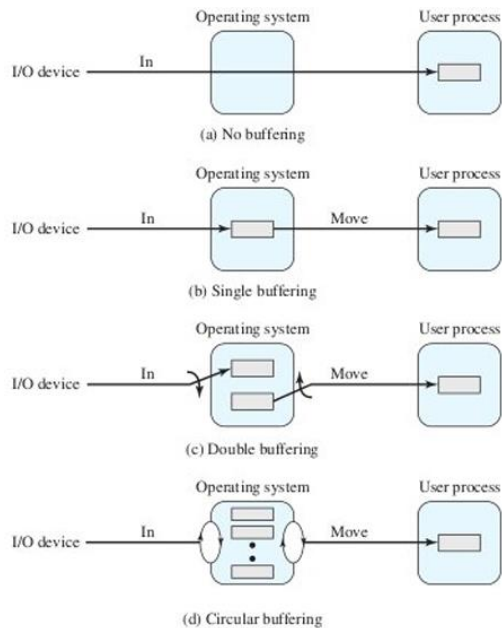
Izvođenje ulaznih prenosa pre nego što se zahtevi postave. – Ako smo tražili blok 76, verovatno će nam biti potreban i blok 77 pa se I on učitava.

Izvođenje izlaznih prenosa neko vreme posle postavljanja zahteva - Ako pritisnemo ctrl+s on kaže da je momentalno sačuvano (iako je ogroman fajl). On je već uradio tu operaciju I stavio u bafer i kada kliknemo samo prebaci na disk.

Vrste periferala:

- **Uređaji orijentisani na blokove** – skladište ceo blok podataka
- **Uređaji orijentisani na tokove** – skladište samo posebne bajtove

Vrste ulaznih bafera:



Jednostruki bafer

Učitavanje unapred, tj. predviđeni ulaz.

Kada se završi obrada, blok podataka se učitava u bafer za većinu zadataka. Ovo omogućava operativnom sistemu (OS) da razmesti ili zameni (swap) procese jer se baferi nalaze u sistemskoj memoriji, što znači da procesi ne zauzimaju dragoceni prostor u svojoj vlastitoj memoriji za te bafer podatke.

Dvostruki bafer

Operaciji se dodeljuju dva sistemska bafera

Proces prenosi podatke u jedan bafer (ili iz njega), a operativni sistem prazni (puni) drugi bafer.

Kružni bafer

Dvostruko baferovanje može da podbaci ako process izvodi brze neprekidne nizove U/I operacija.

Kružni bafer – zbirka više bafera.

Isto kao dvostruko bafer, ali ima više bafera koji se smenjuju.

3. Optimizacija performansi diska

Vreme pristupa – Vreme koje protekne od trenutka kada je pisanje/čitanje zahtevano do trenutka kada transfer započne

Vreme pretrage – Vreme potrebno za pozicioniranje glave čitača

Rotaciono kašnjenje – Vreme potrebno da se traženi sektor nađe ispod glave

Vreme prenosa (stopa prenosa podataka) $T = (b/rN)$. r – rpm, b – broj bajtova za prenos, N – broj bajtova po stazi.

Srednje vreme otkaza

Ukupno vreme pristupa

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

gde je T_s prosečno vreme pozicioniranja.

4. Politike raspoređivanja diska – FIFO, PRI, LIFO, SSTF

FIFO – najjednostavniji oblik, poštuje svaki zahtev, nema izgladnjivanja. Prihvatljivo ako sistem radi sa jednim procesom (DOS operativni sistem), loše performanse u slučaju da ima više procesa

PRI – prioritetno. Cilj nije optimizacija, već ispunjavanje nekog drugog zahteva. Obično prednost imaju kratki i interaktivni poslovi

LIFO – najskoriji zahtev ima prednost pri procesiranju. Cilj je poboljšanje lokalnosti, jer najskoriji zahtev obično najmanje pomera glavu diska. Može doći do gladovanja

SSTF – Shortest Service Time First, bira zahtev za najmanje pomeranje glave, ako glava ide u oba smera isto, bira se slučajnim izborom.

5. Politike raspoređivanja diska – SCAN, C-SCAN, N-SCAN, F-SCAN

SCAN – glava se pomera samo u jednom smeru usput ispunjavajući zahteve, zatim se obrće. prednost daje krajnjim stazama i poslovima koji su najkasnije stigli.

C-SCAN – ograničava skeniranje na samo jedan smer i time rešavamo problem uniformisanja dužine čekanja.

N-stepeni SCAN – deli redove zahteva na podredove dužine N . Podredovi se obrađuju jedan po jedan pomoću SCAN, dok se jedan obrađuje, novi zahtevi stižu u neki drugi red.

F-SCAN – ima dva reda čekanja – aktivan red i nov red za zahteve. Swapuju se pointeri kad se jedan red isprazni.

6. RAID 0,1, Redundant Array of Independent Disks

Skup fizičkih uređaja koje sistem vidi kao jedan logički uređaj.

Podaci su raspoređeni preko niza fizičkih uređaja.

Kapacitet se koristi za skladištenje parnosti što garantuje obnovljivost u slučaju otkaza jednog diska.

RAID 0 (stripe) – nije pravi RAID jer ne uključuje redundansu. Ako su performanse i kapacitet glavni parametri, RAID0 je dobar izbor. Veliki dobici u performansama ako se traže velike količine susjednih podataka ili veliki broj konkurentnih zahteva. ako disk otkaze, gubimo podatke- nije baš pouzdan

RAID 1 (mirror) – logička traka se preslikava na dva diska. Čita se sa bilo kog, a piše se na oba. oporavak posle otkaza jednostavan. Kapacitet je manji.

7. RAID 2, 3

RAID 2 – trake su male, bajt ili reč. Radi Hammingov kod za ispravljanje grešaka. Za pisanje i čitanje pristupa svim diskovima, ima primenu kad dolazi do mnogo grešaka diska – a to je skoro nikad.

RAID 3 – slično kao RAID 2, samo sa jednim redundantnim diskom. Umesto složenog koda za ispravljanje grešaka koristi se jednostavan bit parnosti za skup pojedinačnih bitova na istim pozicijama na svim diskovima za podatke. Ako disk otkaze, pristupa se bitu za parnost i podatak se obnavlja. dobre performanse ali samo jedan UI zahtev može da se izvršava istovremeno.

8. RAID 4, 5, 6

RAID 4 – isto što i 3 ali umesto malih traka koristi blokove (veće). Na višekorisničkom sistemu povećava performansu, dok na jedнокorisničkom sistemu povećava performanse ako radimo sa velikim fajlovima, inače ostaju iste. Problem je usko grlo u vidu diska parnosti (upisuje se na taj disk pri svakom upisivanju na bilo koji drugi).

RAID 5 – trake za parnost su raspoređene po svim diskovima (rešenje za problem sa RAID4).

RAID 6 – vrše se dva odvojena proračuna parnosti. Podaci se oporavljaju čak i ako otkazu dva diska.

Softverski raid - FTDISK, MDADM

9. Keš diska

Kada se UI zahtev ne zadovolji iz keš diska, mora da se izbací jedan blok i novi dovede u keš.

LRU – least recently used – najduže u kešu a nije referenciran

LFU – least frequently used – brojač svakom bloku u kešu. problem se javlja kada se brojač retko, ali veliki broj puta referencira (tipa 1000 puta u sat vremena)

Zamena zasnovana na učestanosti

Postoje dve sekcije: nova i stara. U novoj sekciji se brojači ne povećavaju ponovnim referenciranjem a blokove izbacujemo samo iz stare sekcije.

10. U/I sistema UNIX

Svakom pojedinačnom U/I uređaju se pridružuje specijalni fajl (b, c) u direktorijumu /dev.

Postoje dve vrste U/I uređaja- **baferovani i nebaferovani**.

Baferovani prolazi kroz sistemske bafere (sistemski bafer keš i karakter red), dok **nebaferovani** ide direktno preko DMA (između U/I i procesa).

Bafer keš je keš diska. Preko DMA kanala vrši kopiranje iz UI oblasti procesa i obratno. Tri **liste**:

- **lista slobodnih slotova** (svaki slot je veličine sektora na disku)
- **lista uređaja** (lista bafera trenutno pridruženih svakom disku)
- **U/I red drajvera** (lista bafera koji stvarno rade ili čekaju na U/I na određenom uređaju)

Za ubrzavanje pretrage koristi se hash tabela. Model je writer / reader.

U karakter red upisuje UI uređaj, a čita ga proces, ili upisuje proces, a čita karakter red . U oba slučaja model producer/consumer. Kada se znak pročita, on se zapravo briše. FIFO princip

Nebaferovani – smanjuje mogućnosti za straničenje što umanjuje ukupne performanse. UI uređaj vezan sa procesom stoga je nedostupan drugim procesima.

11.Linus Elevator, Raspoređivač po roku

Linus Elevator – slično kao tehnika SCAN. Održava jedan red čekanja (i čitanje i upis) i u njemu drži sortirane brojeve bloka uz određene modifikacije (spajanje istih zahteva). Problem je neprestani dotok zahteva. Takođe, isti prioritet čitanja i upisa (čitanje bi trebalo da bude većeg prioriteta).

Raspoređivač po roku - napravljen kao odgovor na ove probleme. Svaki zahtev ima vreme isticanja (čitanje 0.5s, pisanje 5s). Svaki zahtev se postavlja u Elevator red i odgovarajući FIFO red za čitanje/upis.

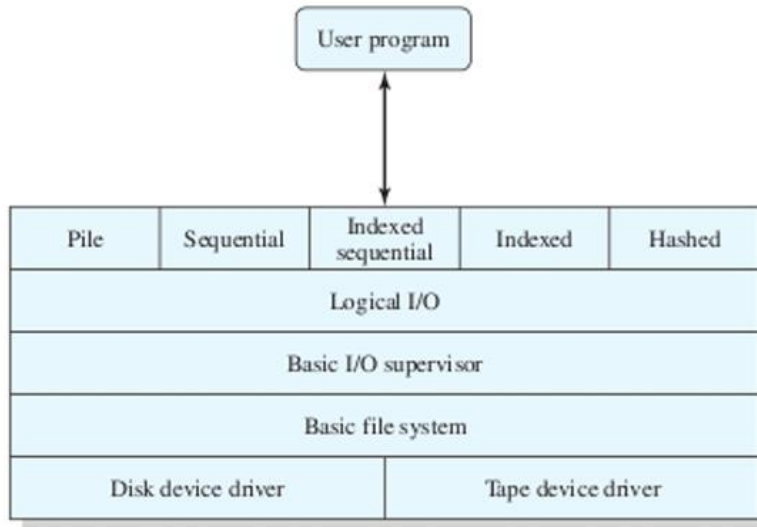
Raspoređivač sa predviđanjem je nadređen raspoređivaču sa rokom. Kada stigne novi zahtev, sistem za raspoređivanje ga zadržava narednih 6ms jer postoje dobre šanse

12. Arhitektura fajl sistema

Drajveri uređaja - komuniciraju direktno sa hardverom.

Osnovni fajl sistem/fizički UI – razmena i baferovanje blokova (deo OS).

Logički UI (radi sa **recordima**, ne blokovima).



13. Organizacija fajlova

Zahtevi fajl sistema su:

- logičko strukturiranje zapisa (records)
- kratko vreme pristupa
- lakoća ažuriranja
- ekonomičnost
- jednostavno održavanje
- pouzdanost

14. Organizacija fajlova – hrpa, sekvencijalni fajl

Hrpa:

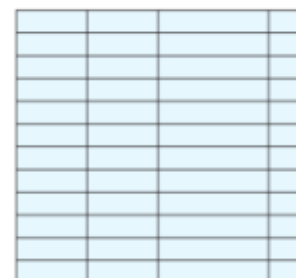
- Najjednostavniji oblik organizacije fajlova
- Podaci se pakuju u redosled u kome stižu
- Posto polja ne stizu redom, svako polje treba da sadrzi opis
- U potrazi za zapisom pretražuje se ceo fajl
- Koristi se kada se cesto menja velicina I struktura fajla

Sekvencijalni fajl:

- Fiksni format
- zapisi iste duzine u odredjenom redosledu
- u fajl idu samo vrednosti polja
- pretrazivanje lose (osim ako ceo fajl nije u RAM)
- Kod dodavanja zapisa, zapis se stavlja u poseban fajl tipa hrpa (*pile*)-**fajl transakcija**, a zatim periodično unose.
- alternativa je organizacija u vidu **povezane liste**.



Variable-length records
Variable set of fields
Chronological order



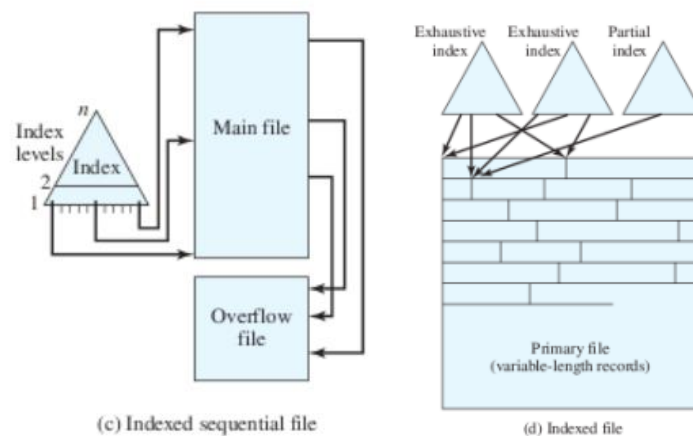
Fixed-length records
Fixed set of fields in fixed order
Sequential order based on key field

15. Organizacija fajlova – indeksirani sekvencijalni fajl, indeksni fajl, direktni (heširani) fajl

Indeksirani sekvencijalni fajl: dodaju se dva svojstva – **indeks fajl** i **overflow file** (fajl prekoračenja). Novi zapis se dodaje u fajl prekoračenja i tu se ažuriraju pokazivači. Indeks se može podići na nivoe.

Indeksni fajl: za svako polje od interesa se može dodati indeks. Iscrpan indeks sadrži po stavku za svaki zapis, dok delimični indeks sadrži stavke za zapise u kojima postoji polje od interesa.

Direktni (heširani) fajl: brz pristup, pristupa se jednom zapisu istovremeno, pristupa sposobnost diska da pristupi svakom bloku sa poznatom adresom.



16. Direktorijumi

Direktorijumi sadrže informacije o fajlovima kao što su atributi, lokacija i vlasništvo.

Operacija: pretraživanje, kreiranje, brisanje, listanje (find), ažuriranje, jedno rešenje je **sekvencijalni fajl** a ako je lista velika koristi se **heširana struktura**.

Prava pristupa: za vlasnika, grupu i ostale.

Direktorijumi imaju prava: citanja (r), pisanja (w), i izvršavanje (x).

	pristupna prava za fajlove	pristupna prava za direktorijume
read (r)	korisnik može pročitati sadržaj fajla, odnosno može prikazivati fajl na ekranu, štampati ga ili kopirati;	korisnik može pročitati sadržaj direktorijuma, što znači i da korisnik može da izvrši komandu ls. Napomena: za prikazivanje detaljnog listinga direktorijuma (ls -l) neophodno je i x pravo nad direktorijumom
write (w)	korisnik može modifikovati sadržaj fajla. Napomena: može obrisati fajl samo ako mu je dato pravo upisa nad roditeljskim direktorijumom;	korisnik može modifikovati sadržaj direktorijuma, odnosno dodavati nove fajlove i brisati postojeće, kreirati i brisati poddirektorijume. Napomena: može obrisati direktorijum samo ako mu je dato pravo upisa nad roditeljskim direktorijumom;
execute (x)	korisnik može izvršavati fajl, pod uslovom da se radi o shell programu ili o fajlu u binarnom izvršnom formatu;	korisnik se može pozicionirati na direktorijum (komandom cd), može prikazivati dugački listing (ls -l) sadržaja i pretraživati direktorijum (find).

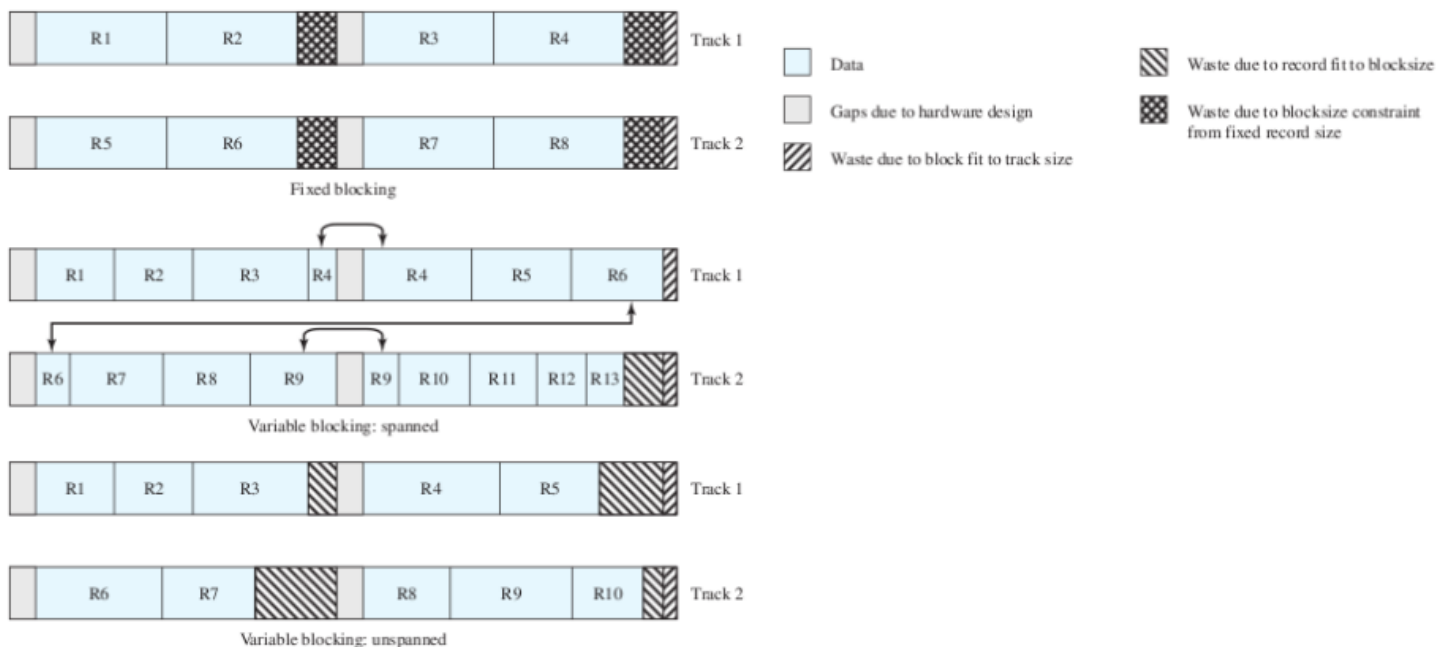
17. Blokovanje fajlova

Blokovanje je pretvaranje zapisa (kao redovi u tabeli bp, records) u blokove koji su osnovna jedinica za rada u fajl sistemu u sekundarnoj memoriji.

Blokovi su vecinom fiksne duzine.

Vrste blokovanja:

- **Fiksno blokovanje** (unutrašnja fragmentacija- neki blokovi nisu skroz puni- neiskorišćen prostor)
- **Promenljive dužine s premošćenjem** (nema neiskorišćenog prostora)
- **Promenljive dužine bez premošćenjem** (postoji neiskorišćen prostor)



18. Modeli dodeljivanja fajlova

Jedinica dodele – jedan blok ili jedan fajl. Politike za dodeljivanje prostora za fajlove su:

- **prvi odgovarajući** (prva nekorišćena grupa blokova dovoljne veličine)
- **najbolji odgovarajući** (najmanja neiskorišćena grupa blokova)
- **najbliži odgovarajući** (najbliža neiskorišćena grupa blokova)

Modeli dodeljivanja fajlova:

- **susedno dodeljivanje** (dodeljuje se određeni broj susednih blokova, dobro za sekvencijalne fajlove, postoji spoljašnja fragmentacija)
- **ulančano dodeljivanje** (na nivou pojedinačnih blokova, nema spoljašnje fragmentacije, nema saglasnosti sa principom lokalnosti pa se povremeno defragmentira)
- **indeksno dodeljivanje** (tabela sadrži poseban indeks u jednom nivou za svaki fajl, dodeljivanje ili fiksne ili promenljive dužine, ako je promenljive dužine defragmentacija smanjuje tabelu)

	Contiguous	Chained	Indexed	
Preallocation?	Necessary	Possible	Possible	
Fixed or variable size portions?	Variable	Fixed blocks	Fixed blocks	Variable
Portion size	Large	Small	Small	Medium
Allocation frequency	Once	Low to high	High	Low
Time to allocate	Medium	Long	Short	Medium
File allocation table size	One entry	One entry	Large	Medium

19. Upravljanje slobodnim prostorom na disku

Tabela bitova

Vektor koji sadrži po jedan bit za svaki blok. ovakva tabela mnogo RAM-a uzima

Ulančani slobodni delovi

Cuvanje informacija o slobodnom prostoru koristeći pokazivace i vrednost duzine svakog slobodnog dela.

Indeksiranje

Slično kao kod fajlova, samo treba da sadrži promenljive delove zbog performansi.

Lista slobodnih blokova

Zadovoljavajući metod, svakom bloku se dodeljuje broj, prostor za tabelu na disku je manji od 1% ukupnog prostora. U ram-u se drži mali deo liste u obliku potisnog steka ili fifo reda.

20. UNIX / LINUX fajl sistem

Vrste fajlova su: regularni, direktorijum, specijalni (dev/), imenovani tokovi (mkfifo), linkovi i simbolički linkovi.

Indeksni čvor je upravljačka struktura koja sadrži ključne informacije koje su potrebne OS za dati fajl (dodeljivanje je dinamičko, po blokovima, bez predviđanja).

Tipovi u okviru OO arhitekture: **Superblok** (monirani fs), **Indeksni cvor** (fajl), **Dentry** (zapis u direktorijumu) i **Fajl** (otvoreni fajl pridruzen procesu)

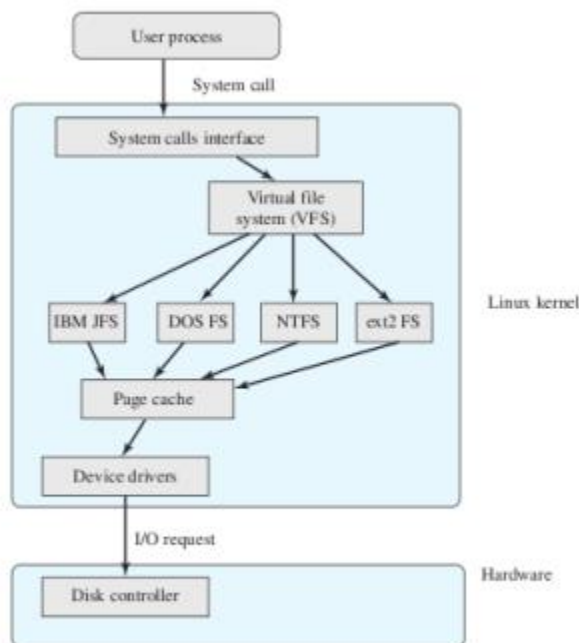


Figure 12.17 Linux Virtual File System Context

21. VSFS (Very simple file system)

4 KB po bloku (fiksna veličina).

Najveći deo uzimaju korisnički podaci, onda imamo i indeksne čvorove koji zapravo čuvaju metapodatke.

Nakon toga, neophodno je pratiti da li su indeksni čvorovi kao i korisnički blokovi slobodni, a za to koristimo još dva odvojena bloka, *i* i *d*. VSFS koristi tabelu bitova za čuvanje slobodnih blokova. Jedan slobodan blok je ostao i njega zovemo superblok i on sadrži informacije o svim fajlovima u sistemu, kao npr broj blokova na disku, koliko indeksnih čvorova postoji, obuhvata i magic number koji predstavlja tip datoteke. Prilikom pokretanja OS prvo se učita superblok pa pomoću njega i ostatak fajl sistema.

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read	read		read				
				read		read				
read()				read			read			
				write						
read()				read				read		
				write						
read()				read						read
				write						

Slika 9: Dostupak bitova fajla na disku

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
create (/foo/bar)		read write	read			read		read		
				read write			write			
write()	read write			read				write		
				write			read			
write()	read write								write	
				write			read			
write()	read write									write
				write						

Slika 10: Kreiranje fajla na disk

22. FFS (Fast file system)

Disk je podeljen u grupe cilindara (zbog poboljšanja lokalnosti).

U svakoj grupi blokova se nalazi kopija superbloka, ib i db (tabela bitova), indeksnih čvorova i podataka. Veličina bloka je 4 KB.

Čuva povezane fajlove na jednom mestu (najčešće ista grupa), a nepovezane odvojeno. Vršiti se amortizacija za čuvanje velikih fajlova (povećavanje veličine bloka). Suportan postupak (smanjenje veličine bloka) može biti izveden i to je radi izbegavanja unutrašnje fragmentacije, odnosno zbog optimizacije.

Fajlovi su mogli biti imenovani drugačije uvođenjem simboličkih linkova, takođe ovde je prvi put uvedena opcija preimenovanja fajla. FFS takođe uvodi ekskluzivno i deljeno zaključavanje fajla.

23. FSCK i vođenje dnevnika

File System Checker radi u 2 režima- interaktivni i neinteraktivni. U neinteraktivnom čini promene za koje je siguran, u interaktivnom ima predloge za izmene i operater mora da odluči koju će akciju izvršiti.

Pregled fsck akcija:

- proveru superbloka (da li je sve logično, ako nije proveru sa kopijama superbloka)
- proveru slobodnih blokova (da li su slobodni zapravo slobodni- da li postoji određeni indeksni čvor koji pokazuje na neki blok iz slobodnih)
- proveru stanja indeksnih čvorova (ukoliko nisu usaglašeni indeksni čvorovi i bitmape, veruje se čvorovima)
- linkovi indeksnih čvorova (hard linkovi – imena fajlova u direktorijima)
- duplikati (dva indeksna čvora pokazuju na isti blok. u tom slučaju, ili se taj indeksni čvor briše ili se blok kopira da svaki indeksni čvor pokazuje na svoj blok.
- pogrešni blokovi (prolazi se kroz listu svih pokazivača i u slučaju da pokazuje na nešto van dometa, pokazivač se briše)
- proveru direktorijuma

problemi kod fsck: jako spor

Journaling

write ahead logging – pre upisivanja novih podataka, zapisati malu napomenu, odnosno log, u strukturu. Ukoliko dođe do pada sistema u toku ažuriranja, uvek možemo pokušati ponovo pomoću beleške iz loga.

Physical logging - predstavlja čuvanje i fizičkog sadržaja u dnevniku. Nakon smeštanja transakcije na disk, vrši se checkpointing, a to je upisivanje podataka umesto starih struktura. Prosleđuju se neophodni parametri i ako se upišu, checkpoint je uspešan.

Kao rešenje problema loših zapisa i nepotpunih recorda u dnevniku, upis se deli na 2 dela – upis u dnevnik kao prvi korak, kao drugi komitovanje dnevnika (dodavanje TxE na kraj cele transakcije) i onda checkpointing.

Oporavak se vrši na sasvim logičan način, u slučaju nepotpunog log fajla, ažuriranje se preskače. U slučaju upisa log fajla, radi se redo logging – ažuriranje celog log fajla (nebitno da li je prethodno upisano nešto ili ne). Doziranje ažuriranja se vrši baferima (ext3 na Linuxu), gde pomoću bafera pakuju transakcije u jednu (kada se nalaze blizu na disku) i onda ih izvršavaju odjednom.

U slučaju nedostatka memorije, koristi se kružni dnevnik sa superblokom. Ubrzavanje procesa se vrši korišćenjem dnevnika bez fizičkih podataka, odnosno samo sa metapodacima.

24. SSD – hardverska arhitektura

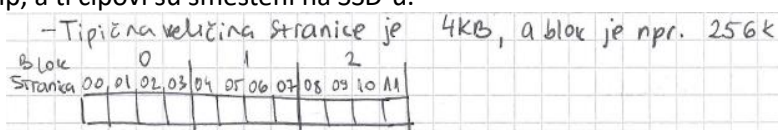
SSD – Solid state drive – kao fles ali ima dodatni kontroler

Postoje 3 nacina da se smesti podatak u celiju:

- **SLC (single level cell)** – 0,1 = 1 bit ili je puno ili prazno
- **MLC (multi level cell)** – 00,01,10,11 = 2 bit zavisi od nivoa napunjenosti
- **TLC (triple level cell)** – 000,001,...,111 = 3bit

Struktura SSD-a:

Najosnovnija jedinica je stranica. Od nekoliko 10-in ili 100-ina stranica se sastoji blok. Blokovi se grupisu u ravan, pa ravan u cip, a ti cipovi su smesteni na SSD-u.



Osnovne operacije flash memorije:

- 1) Čitanje (stranice): Korisnik može čitati bilo koju stranicu. Mogućnost pristupa bilo kojoj lokaciji istom brzinom (random access) – uređaj sa slučajnim pristupom.
- 2) Brisanje (bloka): Ne može da se obriše pojedinačna stranica, nego mora da se obriše ceo blok. Ako hoćemo da obrišemo stranicu moramo biti sigurni da smo sve ostale stranice iz tog bloka upisali u neki drugi blok jer se ceo blok briše.
- 3) Programiranje (stranice): Kada se blok obriše, komanda za programiranje se može koristiti za promenu odgovarajućih bitova u okviru stranice sa 1 na 0. Na taj način se upisuje željeni sadržaj na fleš.

- Na početku se stranica nalazi u stanju INVALID - Brisanjem bloka unutar kojeg se stranica nalazi, stanje stranice postaje ERASED, kao i ostalih stranica unutar bloka i to ih čini programabilnim. Nakon što se stranica isprogramira stanje postaje VALID i može da se čita.

Performanse:

Koliko ciklusa programiranja i brisanja može da se izdrži, koliko je vreme čitanja, vreme programiranja, vreme brisanja. Najbolji je SLC. Ali problem je wear out - dolazi do viška naelektrisanja, pa teško može da se razazna šta je 0, a šta 1. Još jedan problem - kada programiramo jedan blok, bitovi prelaze u susedne - disturbance.

25. SSD – metode FTL mapiranja

FTL – Flash transition layer – prihvata zahteve za pisanje, citanje i pretvara ih u operacije nizeg nivoa.

FTL organizacija: los pristup, direktno mapiranje

- **Direktno mapiranje** – fles je sporiji, kada upisuje puno malih fajlova on za svaki mora da radi **write amplification** (*kada upisujemo jednu stranicu raidmo i brisanje bloka i programiranje*)
- **Log-strukturirani FTL** – ne posmatra SSD kao RAM vec kao log strukturu. To znaci da se podaci samo pakuju a brisanje se izbegava.
- **Wear-leveling** – mora da se void racuna da se svi blokovi i stranice ravnomerno koriste.

FTL mapiranje:

Moze da se desi da nestane struja npr. Tabela se nalazi u RAM memoriji u SSD-u i kada nestane struja tabela se obrise i onda imamo podatke sa kojima ne znamo sta cemo jer nisu alocirani. Ne znamo kojoj logickog stranici pripadaju. Zatos vi SSD-ovi to rezervisu na hardverski nacin. To su spare-area.

Implementiraju se dva nacina:

- **Susedne celije sa podacima**
- **Razdvojene celije sa podacima**

One cuvaju informacije o mapiranju svake strane.

26. SSD – skupljanje otpada, blok i hibridno mapiranje

Sakupljanje otpada:

Premesta korisne podatke i sakuplja smece. Npr. Ako imamo u jednom bloku 4 stranice i u sledecem 2 iz prvog bloka hocemo da obrisemo 2 stranice, onda **garbage collector** premesti ostale 2 stranice zajedno sa stranicama iz 2.bloka, pa moze da obrise ceo 1.blok.

Kada preopstane malo prostora tada se zove garbage collector sto usporava. Npr. Kada kupujemo SSD od 240GB tu sigurno ima 300GB jer je flash memorija jeftina i na taj nacin se obezbedjuju bolje performance.

Problem je kada imamo veliku tabelu mapiranja, to se resave tako sto se radi **mapiranje po bloku** (a ne po stranicama). Znac u tabeli se upisuje 1 zapis po bloku.

Logicke adrese svih stranica prevode se u istu fizicku adresu pocetka bloka, a prepoznajemo ih po ostatku koji dalju pri deljenu sa adresom bloka. **Adresa logickog bloka** dobija se kada se adresa logicke stranica podeli sa brojem stranica po bloku (2002/4 pripada log. Bloku 500)

Ako hocemo neku **stranicu da izmenimo**, treba da procitamo ceo blok, obrisemo sledeci slobodan blok i u njega upisemo se procitane informacije sa izmenom. Opet imamo **amplifikaciju** ako mozda nije bilo potrebe. Zato se koristi **hibridno mapiranje**.

Hibridno mapiranje:

Tabela za mapiranje se razdvaja na 2 tabele – **log i data**.

Data tabela cuva blokove, a log stranice.

Garbage collector gleda sta moze da merge-uje, koje sve pojedinačne adrese stranica moze da spoji i da pretvori u adresu bloka.

27. Vrste procesorskog raspoređivanja (dugoročno, srednjeročno, kratkoročno, U/I), skup relevantnih kriterijuma

Dugoročno raspoređivanje - odluka da se dati proces doda skupu procesa za izvršavanje

Srednjeročno raspoređivanje - odluka da se dati proces doda broju procesa koji su delimično ili potpuno u RAM memoriji

Kratkoročno raspoređivanje (dispečer) - odluka koji od raspoloživih procesa će da se izvršava

Raspoređivanje U/I - odluka koji od nerešenih U/I zahteva će raspoloživi U/I uređaj da opsluži

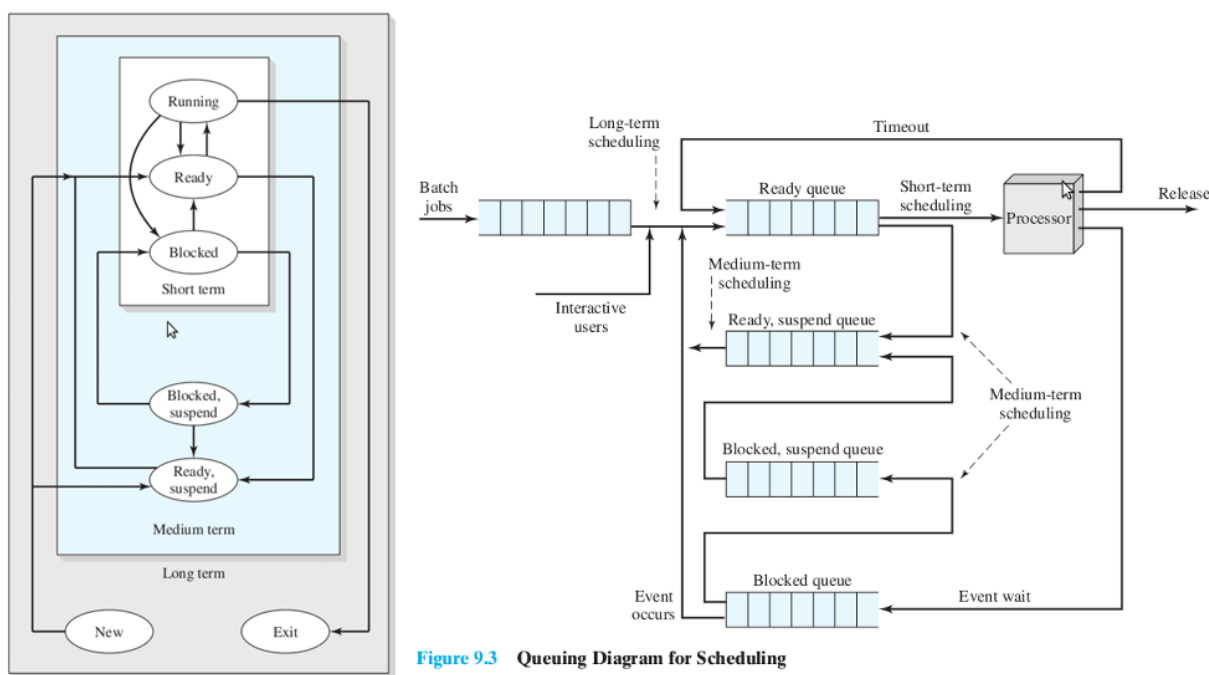


Figure 9.3 Queuing Diagram for Scheduling

Kriterijumi po kojima se procenjuje kvalitet algoritma raspoređivanja:

- orijentisani ka korisniku: vreme prolaska zadatka, odzivno vreme, rokovi, predvidljivost
- orijentisani ka sistemu: propusna moć (br poslova u jedinici vremena), iskorišćenje procesora, pravičnost, primena prioriteta (da ne dođe do gladovanja), uravnoteženje resursa

Funkcija izbora procesa - zavisi od w (vreme provedeno u sistemu u čekanju i izvršavanju), e (ukupno vreme u izvršavanju), s (ukupno vreme usluživanja koje zahteva proces)

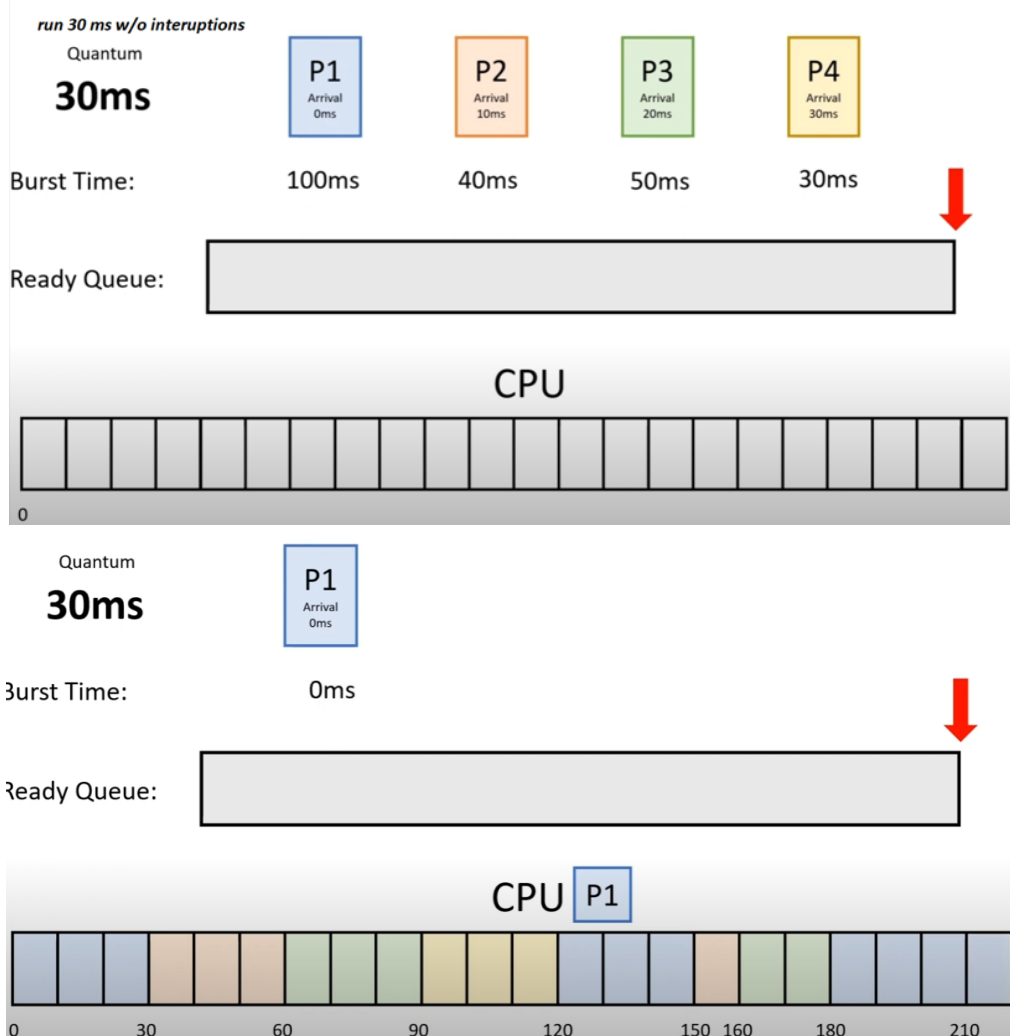
Režim izbora određuje vremenske trenutke u kojima se izvršava funkcija izbora:

- bez prekidanja
- sa prekidanjem

28. FCFS, RR

FCFS (first come first serve) – fifo, algoritam raspoređivanja (favorizuje duge u odnosu na kratke procese, isto one sa intenzivnim korišćenjem CPU u odnosu na one sa u/i operacijama)

Round Robin raspoređivanje – prekidanje zasnovano na signalima clock-a, time slicing algoritam.



29. VRR, SPN

Virtual Round Robin i dalje ima kao prioritet procese sa intenzivnom upotrebom CPU u odnosu na procese sa upotrebom U/I. Dodaje se pomoćni red koji čuva procese koje otpušta neki U/I i on ima prednost u raspoređivanju.

Shortest Process Next: politika bez prekidanja, najkraće vreme usluživanja. problem je procena vremena obrade svakog procesa. moguće gladovanje

30. SRT, HRRN, FeedBack raspoređivanje

Shortest Remaining Time - prekidna verzija SPN algoritma. Bira se proces sa najkraćim preostalim vremenom obrade. Problem je što zapisivanje vremena izvršavanja opterećuje sistem.

Highest Response Ratio Next - minimizuje normirano vreme prolaska, pravično je jer uzima u obzir starost procesa, favorizuju se kraći procesi, ali i duži koji čekaju stare. neophodna procena usluge kao za SRT i SPN.

Feedback raspoređivanje – u slučaju da nema informacija o dužini procesa, u obzir ne dolaze ni SPN, SRT, ni HRRN. kažnjava procese koji se dugo izvršavaju. koristi više redova, svaki proces dolazi u prvi red, pa što više puta bude u procesoru, spada na, po prioritetu, niže redove.

31. Fairshare raspoređivanje

Može da grupiše procese i da dodeljuje težinu procesu/grupi koja definiše udeo u iskorišćenju ukupnih resursa. Težine se računaju na duži rok. Za računanje težine u obzir se uzima prioritet procesa koji zavisi od njegovog skorašnjeg korišćenja procesora kao i skorašnje korišćenje od strane grupe kojoj proces pripada. Svodi se na kažnjavanje procesa koji su se dugo izvršavali jer im prioritet vremenom pada.

32. UNIX raspoređivanje (dodaj)

Računa se prioritet procesa, faktor i komanda na linux-u za fino podešavanje prioriteta je nice.

33. Multiprocesorsko raspoređivanje - statičko i dinamičko, multiprogramiranje pojedinačnih procesora

Statičko dodeljivanje je trajna dodela procesa procesoru od početka do kraja, namenski red čekanja za svaki procesor. **Prednost** je manja režija u zameni procesora, a mana što procesor može da bude besposlen. Donekle se rešava zajedničkim redom čekanja.

Dinamičko dodeljivanje znači da se niti mogu pomerati između redova čekanja radi uravnoteženja opterećenja. Linux koristi ovakav pristup

Modeli **schedulinga**

master-slave (na jednom procesoru kernel i scheduler, na ostalima korisnički procesi)

ravnopravni (svi procesori mogu da izvršavaju i korisnički i kernel/scheduler kod)

34. Raspoređivanje niti - deljenje opterećenja, grupno raspoređivanje, namenska dodela procesora, dinamičko raspoređivanje

Deljenje opterećenja – svaki besposleni procesor bira nit iz zajedničkog reda. Ravnomerna podela opterećenja, raspoređivač se izvršava na trenutno slobodnom procesoru. Mane su što pristup centralom redu čekanja mora da se izvrši preko mutexa, što je u sistemima s mnogo procesora usko grlo. Nije verovatno da će se obrada prekinutih niti nastaviti u istom procesoru, malo je verovatno da će grupa niti iz aplikacije dobiti dobre procesore istovremeno.

Grupno raspoređivanje – skup povezanih niti se istovremeno raspoređuje na više procesora. Režija se smanjuje jer jedna odluka utiče na više procesa/niti. Mana je da je moguće da neki procesori ostanu besposleni.

Namenska dodela procesora – procesu se dodeljuje skup procesora jednak njegovom broju niti. Koristi se kada imamo puno procesora – ekstremni oblik grupnog raspoređivanja. Broj niti ne bi trebalo da prelazi broj procesora u sistemu.

Dinamičko raspoređivanje – broj niti u procesu se menja u toku izvršenja procesa. I OS i aplikacija su uključeni u donošenje odluke o raspoređivanju. Prednosti ovog pristupa ruše kompleksnost i cena režije.

35. Raspoređivanje u realnom vremenu, klasifikacija zadataka, osobine real-time sistema, statičko i dinamičko raspoređivanje

Def

Tacnost rezultata ne zavisi samo od logickog rezultata racunanja, vec i od vremena u kome je rezultat proizveden.

Npr. Robot je zaduzen za proizvodni proces, mora da montira neki deo na auto na proizvodnoj traci, on mora to da izvrši u nekom vremenskom ogranicenju, ako to ne obavi u tom vremenskom ogranicenju auto ce da se pomeri i doći ce do materijalne stete a može doći i do ljudskih zrtava. Nije bitno da se zadatak obavi sto pre vec da se ne omasi rok. Neki od zadataka imaju izvestan **stepen hitnosti**.

Vrste zadataka:

- Cvrst zadatak – rok mora da se ispostuje
- Labav zadatak – ispunjenje roka je pozeljno
- Aperiodican zadatak
- Periodican zadatak

Osobine real-time sistema:

- **Determinizam** – Mera ove sposobnosti je vremenski interval od pojave prekida sa uredjaja visokog prioriteta do pocetka opsluzivanja.
- **Odzivnost** -Koliko je potrebno OS-u da posle potvrđivanja prekida izvrši ISR (inijalizacija, izvršenje ISR, Efekat ugnjezdjena prekida)
- **Korisnicko upravljanje** – Pravljenje razlike izmedju cvrstih i labavih zadataka i odredjivanje prioriteta
- **Pouzdanost** – Smanjenje nivoa usluge umesto potpunog otpusta
- **Operacija postepenog otkaza** – Odrzati konzistentnost po svaku cenu

Kratkorocni scheduler sistema u realnom vremenu ima za cilj da ispostuje sto vise cvrstih zadataka i da prodje sto vise labavijih

Staticki pristup:

- **Pomocu tabela** - Primenjivo na periodicne zadatke, jer znamo kad sta treba da se desi. Razvija se raspored koji omogucava izvršenje svih zadataka. Jedna od metoda je posao sa *najranijim rokom prvi*.
- **Prekidno rasporedjivanje na osnovu prioriteta** – Za zadatke koji su periodicni i zna se u kom vremenu dolaze i koliko traju može da se izracuna koliko ce da koriste procesor, na osnovu toga se pravi sema prioriteta i svakom od tih procesa se dodaje prioritet na osnovu karakteristika (koliko cesto dolazi, koliko traje, koliko koristi UI).

Dinamicki pristup:

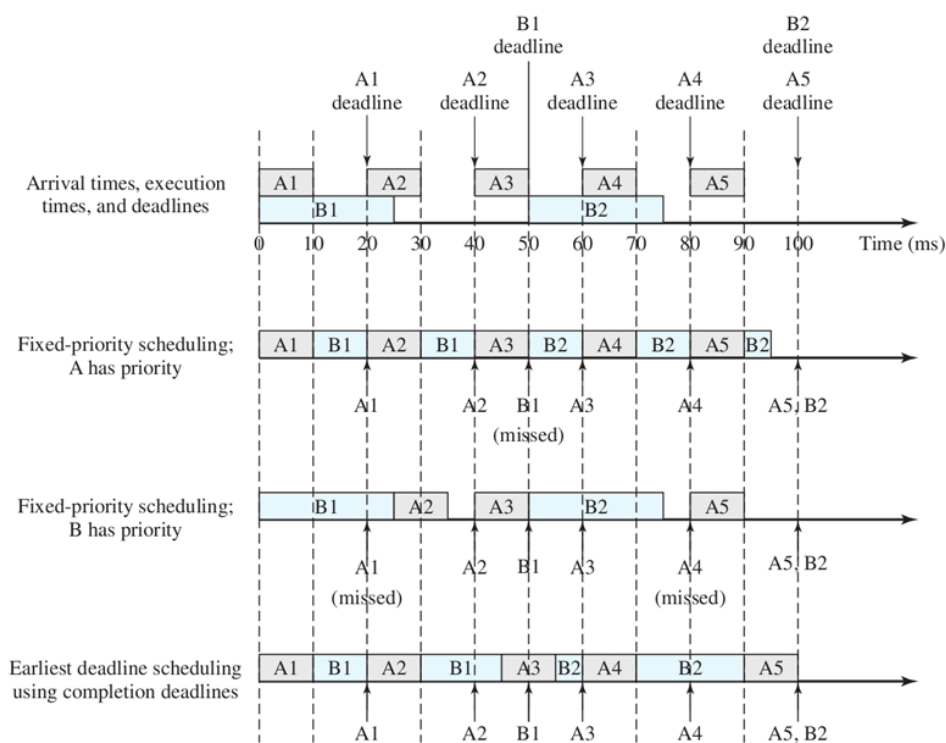
- **Rasporedjivanje zasnovano na planiranju** – Kada posao stigne, pravi se raspored koji poštuje sve rokove. Za aperiodične zadatke, fleksibilan.

- **Rasporedjivanje „najbolje sto moze“** - Koriste ga danasnji OS, OS daje prioritet na osnovu karakteristika, postoji mogućnost promašaja, ali fleksibilno za aperiodične zadatke

36. Periodični zadaci - Raspoređivanje sa rokovima završetaka

Statički pristup određivanja prioriteta nije koristan. Neophodno je da se koristi dinamički, gde se u svakom novom time slajsu (timeslice, intervalu kad procesor bira novi proces) uzima proces najvišeg prioriteta. Prekidni scheduler, periodični zadaci

A i B su periodični zadaci, mora da se pazi na svaki deadline. Ako A ima prioritet on ce ispostovati svoje deadlineove ali B nece. Ako se prvo pocne B, posto je duzi odmah u startu ce da se omasi deadline za A. Koristi se sema zvana **Earliest deadline scheduling using completion deadlines**. Ako imamo informacije o deadlineovima onda dinamički određujemo prioritet.



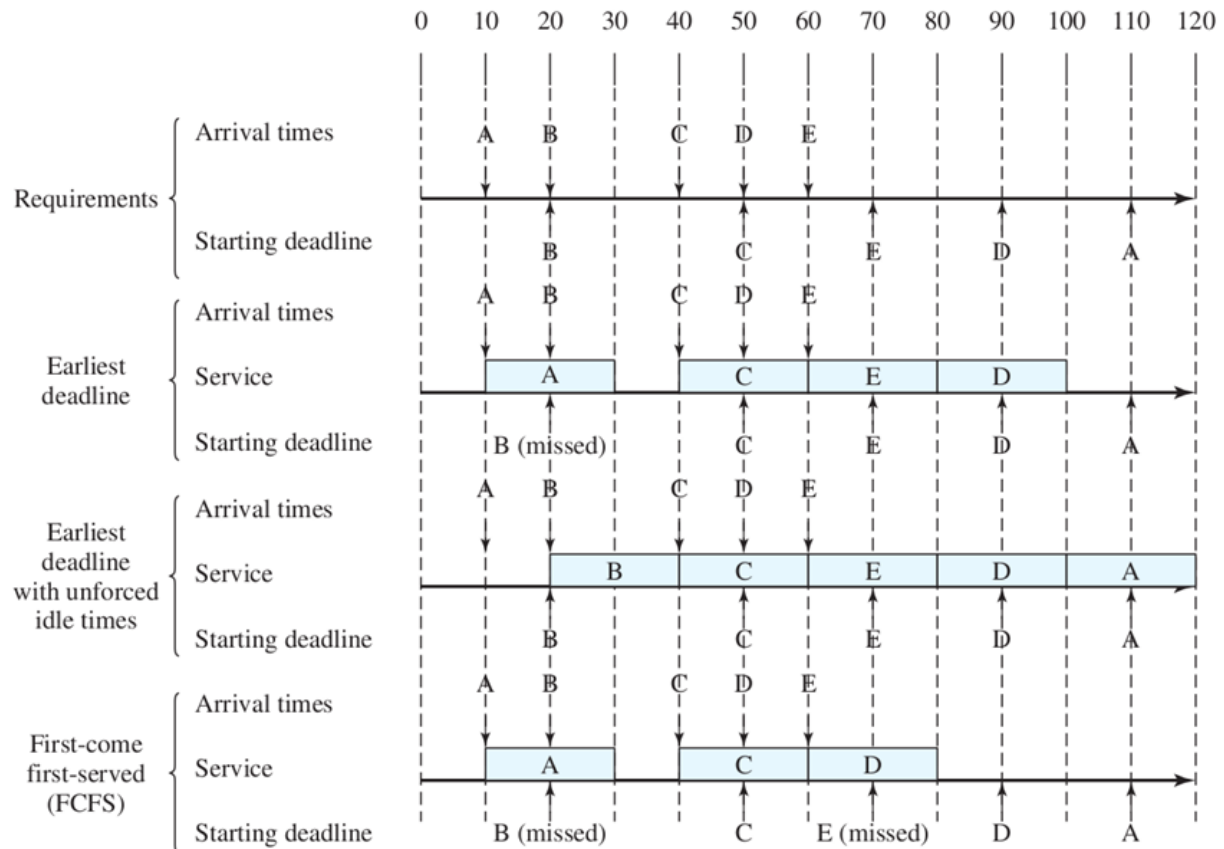
37.

Aperiodični zadaci - Raspoređivanje sa rokovima zapocinjanja

Neprekidni scheduler, rokovi zapocinjanja su ostavljeni da označe kada proces treba da krene sa izvršavanjem a ceo problem predstavlja uklapanje svih procesa u željeno vreme izvršavanja.

Ako za neki proces imamo da je rok zapocinjanja daleko onda je to nizi prioritet od pocesa ciji je rok zapocinjanja odmah (najvisi prioritet), u tom trenutku procesor mora biti slobodan.

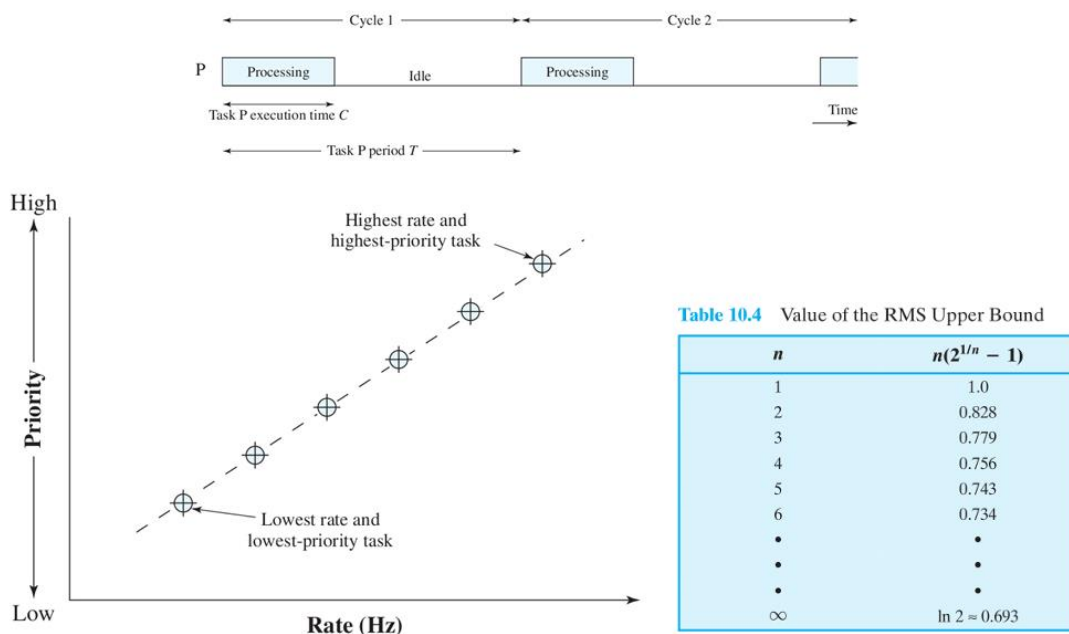
Earliest deadline with unforced idle times – Ne prisiljavamo procesor da radi nesto, ako bas ne mora, bolje da bude slobodan. Ako imamo proces kome je rok mnogo daleko, bolje da uopste ne aktiviramo proces u tom trenutku, nego kasnije kad se uoci prilika.



38. Periodični zadaci – Rate Monothonic Scheduling (RMS)

Spada u statičke šeme i dosta se koristi u industriji.

Za RMS, zadatak sa najvišim prioritetom je onaj s najkraćim periodom dolaženja, zatim onaj sa sledećim najkraćim periodom itd.



39. Inverzija prioriteta

Def (inverzija prioriteta)

Pojava kada okolnosti unutar sistema prisiljavaju zadatak viseg prioriteta da ceka zadatak nizeg prioriteta

Def(neogranicena inverzija prioriteta)

Pojava u kojoj trajanje inverzije prioriteta zavisi ne samo od vremena zahtevanog za rukovanje sistema deljenim resursom, vec i od nekih drugih spoljnih okolnosti.

Resenje (nasledjivanje prioriteta)

Zadatak nizeg prioriteta nasledjuje prioritet zadatka sa visim prioritetom koji trazi resurs koji oni dele.

Drugo resenje je vrhunski prioritet. Proces koji zakljucava resurs uzima prioritet koji je najvisi.

40. Klijent-server aplikacije, ispravno i neispravno korišćenje arhitekture, klase klijent/server aplikacija, troslojna arhitektura, konzistentnost keša

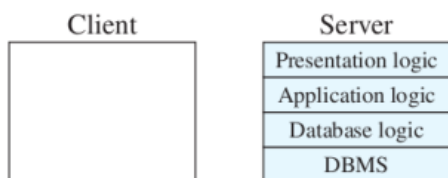
Neispravno korišćenje – fat client, baza se nalazi na serveru i klijent joj pristupa korišćenjem upita, bez ikakve dodatne logike i obrade na serveru.

Neophodno je optimizovati komunikaciju između klijenta i servera.

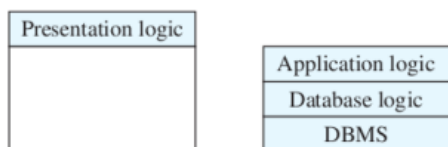
Primer ispravnog koriscenja: Saljemo vise upita I dobijamo odgovor (npr select count(*), select * where...). Server nam vrati npr 100,100000, 1 red...

Primer nepravilnog koriscenja – Trazimo npr select * from .. where .. i u where stavimo filter koji je definisan siroko I onda se na klijenta vrati 30000 redova npr, treba optimizovati.

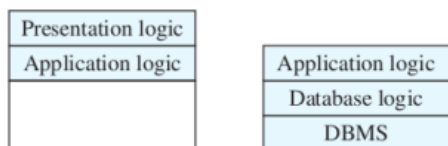
Klase aplikacija



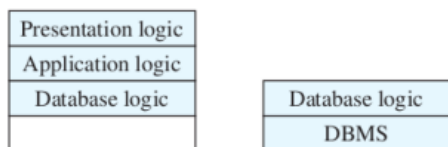
(a) Host-based processing



(b) Server-based processing



(c) Cooperative processing



(d) Client-based processing

Troslojna arhitektura podrazumeva ubacivanje dodatnog sloja koji se zove middleware i ona služi kao posrednik između klijenta i servera.

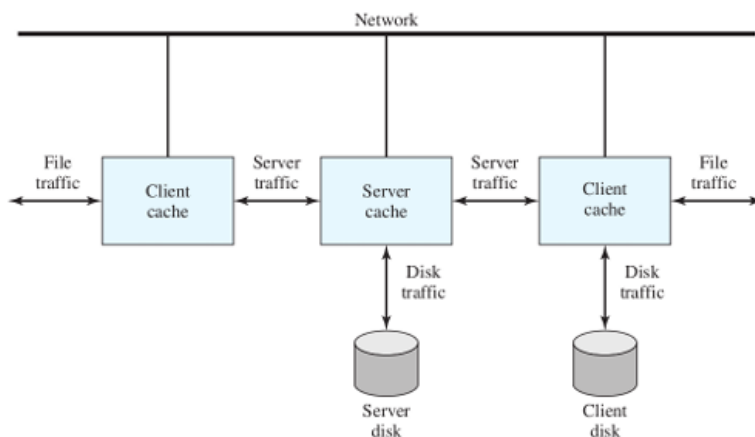
Keš se kupi i na klijentskom i na serverskom računaru i ima istu svrhu kao svaki drugi keš.

Zahtev se prvo prosledjuje kesu klijenta ("saobraćaj fajla"). Ako se tamo ne zadovolji, prosledjuje se lokalnom disku ("saobraćaj diska"). Ako ga ni tamo nema, onda se zahtev prosledjuje server ("saobraćaj servera") koji ima i svoj kes.

Resenja za **konzistentnost** kesu:

Tehnika lokovanja fajlova, konzistentnost po ceni performance

OS Sprite – Obavestavanje procesa putem poruka o nedostatku konzistencije



41. RPC (Remote Procedure Call), klasteri

RPC predstavlja pristupanje uslugama koje su stacionirane na drugim sistemima. Korisnik tog poziva može da bude svestan implementacije ali i ne mora.

Klasteri su skup međusobno povezanih potpunih računara koji rade zajedno kao ujedinjeni resurs stvarajući iluziju da je to jedan računar. Apsolutna i rastuća skalabilnost, visoka raspoloživost, bolji odnos cena i performanse.

Kategorizacija klastera:

- **pasivni standby** – drugi server radi kad prvi otkáže, lako se implementira, ali je skupo jer se drugi server ne koristi za procesiranje podataka
- **aktivni standby** – drugi server se koristi i za rad, ali je kompleksnija implementacija od pasivnog standby principa

Druga kategorizacija:

- **separate servers** (odvojeni serveri imaju svoje diskove, podaci se u kontinuitetu kopiraju sa jednog na drugi server, pristupnost je velika, ali aktivnost(opterećenje) takođe zbog stalnog

kopiranja, rsync- command-line tool za transfer fajlova i direktorijuma na lokalne i udaljene lokacije)

- **servers are connected to disks** (serveri su spojeni na iste diskove, ali svaki koristi svoj. u slučaju otkaza servera, drugi server preuzima njegove diskove. smanjeno opterećenje mreže zbog manje kopiranja. zahteva disk mirroring ili RAID za slučaj otkaza diska)

- **servers share disks** (više servera deli pristup disku, prednosti su slabo zagušenje/opterećenje i smanjen rizik pada servera, ograničenja su to što zahteva lock manager softver. uglavnom se koristi sa disk mirroringom ili RAID).

Karakteristike:

- jedinstvena ulazna tačka
- jedinstvena fajl hijerarhija
- jedinstveno mesto upravljanja
- jedinstveno virtuelno umrežavanje
- jedinstven job manager, npr Torque
- jedinstveni UI prostor
- jedinstven prostor procesa
- preseljenje procesa

42. i 43. Virtuelizacija - osnovni koncepti, prednosti i mane, kategorije, hipervozor

Iluzija da je neki koncept unutar računara poseban entitet.

Prednosti virtualnih mašina:

- brza skalabilnost (pravljenje i kopiranje VM iz određenog imidža)
- uživo migracija (prebacivanje VM na drugi server, čišćenje i održavanje prvog, pa vraćanje mašina na stari server bez ikakvog downtime-a)
- balans opterećenja (load balancing)

Mane VM: (marginalne)

- VM mogu imati ograničenja u vidu RAM memorije i CPU

Tehnike virtualizacije:

Puna virtualizacija

Sve se do detalja emulira. Komplikovano je i bacanje resursa

Paravirtuelizacija

Hardversko okruženje nije simulirano. drajveri za „virtualne uređaje,,.

Hardverski asistirana virtualizacija

Nova tehnika, cpu i ostale komponente (najčešće na serverima) su pripremljene da na njima ne radi glavni os već manje više guest (VM) operativni sistemi.

Hipervizor (VM Manager) je osnovna komponenta kod virtuelizacije i on omogućava većem broju operativnih sistema da se izvršavaju konkurentno na istom računaru.

Postoje dva tipa:

- tip 1 je nezavistan od operativnog sistema i on je sam sistem za sebe. Na njemu mogu da rade isključivo virtuelne mašine
- tip 2 je kao virtual box. on se instalira unutar postojećeg operativnog sistema i radi na osnovu operativnog sistema na kom je instaliran

U današnje vreme se najviše koristi KVM (kernel virtual machine) hipervizor.

44. Cloud computing - osnovni koncepti i kategorizacije

Cloud computing je model računarstva koji omogućava pristup računarstvenim resursima (kao što su računari, mrežna infrastruktura, skladište podataka, softverske usluge itd.) putem interneta. Ovo omogućava korisnicima da koriste te resurse bez potrebe za posedovanjem fizičkog hardvera ili infrastrukture.

Podela po vlasnistvu resurs:

- **Javni resursi** – dostupni klijentima iz celog sveta
- **Privatni** – unutar velikih kompanija ili vladinih usluga(npr. Comtrade ima svoj cloud jer zele svoje podatke da zadrze kod sebe)
- **Hibridni** – najcesce private kompanije koje ustupaju visak resursa da bi isplatile sopstvene troskove
- **Community** – nekoliko firmi koriste iste CC resurse

Delimo Cloude prema nacinu isporuke:

- **Aplikacije** – Korisnik ispred sebe ima aplikaciju koju koristi kao standardni desktop korisnik. (*SaaS*) (*google docs, office 365*)
- **Platforma** – pružaju se usluge u vidu servisa, a od tih usluga tj. komponenti mozemo praviti neke druge komponente. Npr. Servis za autentikaciju koji je na cloud-u mozemo da pozivamo preko nekog API-a. (*PaaS*)
- **Infrastruktura** – direktno se kupuju virtuelne masine, block storage, tj. drajvovi I mrežni kapaciteti. (*IaaS*)

IaaS – umrezavanje, skladištenje, server i virtuelizacije – to nam je obezbedjeno kada zakupimo virtuelnu masinu, a onda sami instaliramo dalje OS.

PaaS – kao wordpress koji smo koristili preko Azura. Dobili smo samo nase podatke i aplikaciju.

SaaS – kompletni softveri, nas nista ne interesuje

45. Docker kontejneri - osnovni principi. Copy on Write, poređenje sa VM

Problem isporuke aplikacija – napravili smo aplikaciju koja kod nas radi, ali vrlo lako moze da se desi da ne radi na drugom mestu.

Za isporuku aplikacije nije pogodno koristiti VM, za to se koristi kontejner koji zadržava lepe karakteristike VM.

Docker je platforma koja pojednostavljuje proces građenja, pokretanja, održavanja i distribuiranja aplikacija. Radi po principu virtuelizacije na OS-nivou. Kontejneri su instance docker imidža (imidž

je niz instrukcija koji prave kontejner, kao templejt, liči na snapshot u VM okruženjima). Svaki docker sadrži linux kernel kao osnovu i na njoj gradi sve aplikacije koje će kontejner sadržati.

Copy on write – sve vreme dok se zahteva pristup za citanje radi se sa istim pointerom, a kada se traži zahtev za upis, pravi se kopija podataka.

46. Vrste pretnji računarskim sistemima

Zahtevi bezbednosti su:

- poverljivost – čitanje podataka
- integritet – menjanje podataka
- raspoloživost – pristup podacima
- autentičnost – lažno predstavljanje

Vrste pretnji:

- **Prekid** - raspoloživost
- **Presretanje** - poverljivost
- **Menjanje** - integritet
- **Fabrikacija** – autentičnost

Napad se može vršiti na hardver, softver, podatke, komunikacione linije.

Napadi se dele na:

- **Pasivne** - teško se otkrivaju jer ne menjaju podatke. najefikasnija je odbrana šifrovanjem (primer: analiza saobraćaja, preuzimanje sadržaja poruke)
- **Aktivne** – uključuju bar neku izmenu sadržaja ili toka podataka. Maskiranje, ponavljanje, izmena poruke, odbijanje usluge

47. Vrste i karakteristike malware-a

Vrste su:

- **Maskirač** – pojedinac koji probija sistemsku kontrolu kako bi iskoristio validan korisnički nalog
- **Insajder** – zvanični zlonamerni korisnik
- **Prikriveni** korisnik – uzima kontrolu koju ima supervizor sistema (root).

Napadi se dele na **maligne** i **benigne**.

Fajl sa lozinkama se može zaštititi na dva načina (/etc/shadow) – jednosmerno šifrovanje, kontrola pristupa

48. Zaštita - kontrola pristupa orijentisana na korisnike, podatke

Zaštita memorije se svodi na razdvajanje memorijskog prostora različitih procesa implementiranjem virtuelne memorije. Straničenje ili segmentacija, bez dvostrukih unosa.

Zaštita korisnika se odnosi na zaštitu podataka i koristi se ID/lozinka. U distribuiranom okruženju kontrola može biti centralizovana (LDAP/Kerberos, ActiveDirectory), decentralizovana ili u dva nivoa.

Zaštita podataka se vrši uvođenjem privilegija – subjekat i pravo pristupa. Postoje liste za kontrolu pristupa (za svaki objekat se nabrajaju korisnici i prava pristupa) i kartice mogućnosti (za koje je objekte i postupke korisnik ovlašćen i na koji način- rizično jer kartice mogu da se razmenjuju po sistemu).

49. Otkrivanje napada, primer buffer overrun

Bitna je brzina zbog smanjenja obima štete.

Otkrivanje se statistička nepravilnost ili otkrivanje na osnovu pravila (odstupanje korisnika od ustaljenih šablona).

Osnovno oruđe za otkrivanje napada je **tumačenje nadzornih logova** (subjekat, akcija, objekat, uslov izuzetka, upotreba resursa, vremenska oznaka).

Buffer overrun se javlja kada zauzmemo više memorije nego što smo namenili u programu te program počne da upisuje na neočekivana mesta i počne da se ponaša nestabilno (različiti ishodi su mogući).

50. Praktični primeri mrežnih napada ([link](#))

DHCP Snooping (Dynamic Host Configuration Protocol)

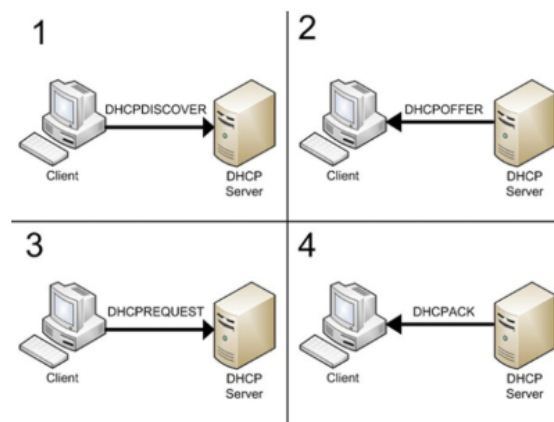
Ovako funkcioniše dhcp:

1. **Faza otkrivanja.** Klijentski uređaj *broadcast*-uje (šalje svima na mreži) *DHCP Discover* poruku u nadi da će pronaći *DHCP* server koji će odrediti njegovu konfiguraciju.

2. **Faza ponude.** Kada *DHCP* server primi *DHCP Discover* poruku, na njega odgovara tako što šalje *DHCP Offer* poruku. Tada server rezerviše jednu *IP* adresu za klijenta. S obzirom da klijent još uvek nema *IP* adresu, server takođe vrši *broadcast* i popunjava polje *clientID* *MAC* adresom klijentskog uređaja kojem je ova poruka namenjena.

3. **Faza zahteva.** Klijent, nakon što mu stigne *DHCP Offer*, *broadcast*-uje *DHCP Request* poruku i na taj način javlja odgovarajućem serveru da želi da koristi njegovu konfiguraciju, ali i ostalim serverima (u slučaju više *DHCP Offer* poruka) da mogu da oslobode rezervisane *IP* adrese.

4. **Faza potvrde.** Server šalje klijentu *DHCP ACK* poruku i daje mu do znanja da je proces konfiguracije gotov i da može da koristi dodeljenu konfiguraciju.



Napad se vrši preko posrednika, odnosno ovaj napad jedan od **man-in-the-middle** napada.

Problem se dešava kada imamo neautorizovani dhcp server na nesigurnom portu i koji presereće gorenavedene zahteve i vrši ceo proces bez našeg odobrenja. Rešava se onemogućavanjem tog porta. Rešava se sa konfiguracijom rutera, ip dhcp relay information trust-all (govori dhcp agentu da prihvati sve pristigle pakete, ako je adresa gatewaya nepoznata u pristiglom dhcp paketu, on će biti odbijen)

Krađa lozinke preko HTTP ili Telnet kanala

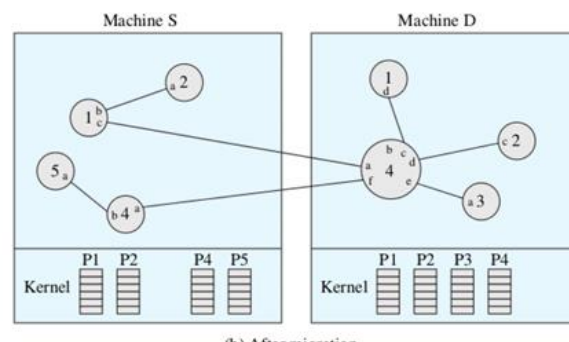
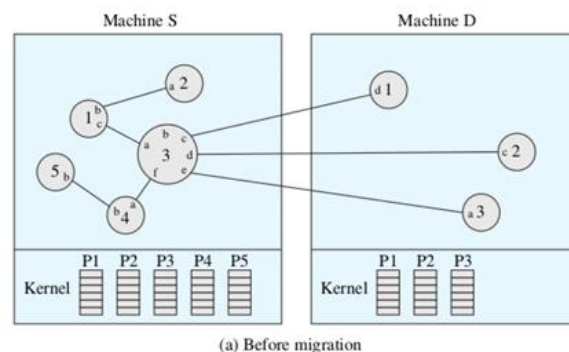
Telnet je jednostavan protokol koji se koristi da osigura korisniku jednog računara sesiju za korišćenje komandne linije na drugom računaru - mana je da sve što se šalje preko mreže ide u tekstualnom formatu. Mogućnost krađe podataka je vrlo velika, uzevši u obzir da u komunikaciji između dva računara se može naći više nepoznatih korisnika. Alternativa tome je ssh, koji kriptuje sve što šalje preko mreže na način koji jedino dva računara koji direktno komuniciraju umeju. Za HTTP je i problem i rešenje sasvim slično - http ne šifrjuje podatke i ne garantuje sigurnu konekciju dok HTTPS šifrjuje sav saobraćaj između servera i klijenta.

51. Migracija procesa

Predstavlja prenos dovoljne količine stanja nekog procesa tako da se taj proces može izvršavati na drugom računaru.

Praktikuje se zbog deljenja opterećenja (load balancing), performanse komunikacija, raspoloživosti, upošljavanja posebnih mogućnosti hardvera/softvera. Mehanizmi migracije:

- **Non-Preemptive migracija** – pre nego što se proces pokrene
- **Preemptive migracija** – u toku izvršavanja procesa

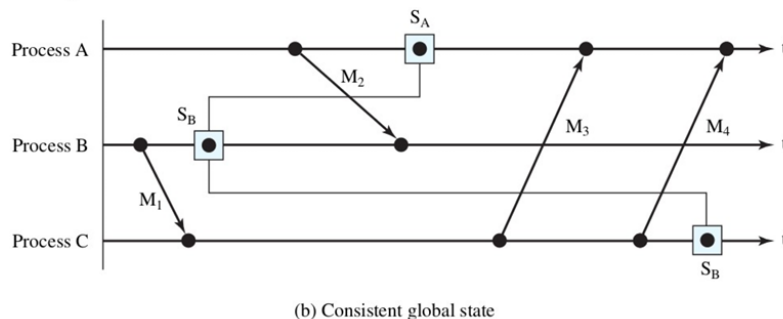
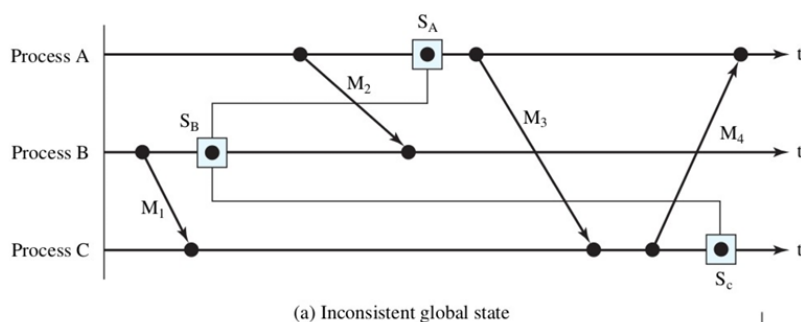


52. Distribuirana globalna stanja – konzistentno i nekonzistentno stanje

Aksiom 1: nemoguće je da jedan OS zna stanja svih procesa u distribuiranom sistemu.

Konzistentno je kada su poruke ili poslate ili primljene.

Ne postoji situacija da je neka poruka poslata a nije primljena.

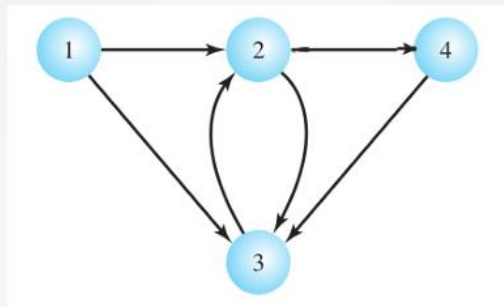


53. Algoritam distribuiranog trenutnog snimka

Specijalna kontrolna poruka je **marker**. Proces p nakon prijema markera radi atomski (kao transakcija), od procesa q:

- p zapisuje sopstveno stanje (niz poruka koje su poslate/primljene duž kanala),
- stanje ulaznog kanala od procesa q kao da je prazno
- p prenosi marker svim susedima duž svih izlaznih kanal

Kada p primi marker, npr od procesa r, on zapisuje stanje kanala od r ka p kao niz poruka koje je primio od zapisa Sp do trenutka prijema markera od r. Algoritam se završava nakon što p primi marker duž svih svojih ulaznih linija. Bilo koji proces može pokrenuti algoritam slanjem markera. Okončava se u konačnom vremenu, distribuiranog je tipa, konzistentno globalno stanje se skuplja u jednom čvoru, ne utiče na ostale distribuirane algoritme.



Process 1 Outgoing channels 2 sent 1,2,3,4,5,6 3 sent 1,2,3,4,5,6 Incoming channels	Process 3 Outgoing channels 2 sent 1,2,3,4,5,6,7,8 Incoming channels 1 received 1,2,3 stored 4,5,6 2 received 1,2,3 stored 4 4 received 1,2,3
Process 2 Outgoing channels 3 sent 1,2,3,4 4 sent 1,2,3,4 Incoming channels 1 received 1,2,3,4 stored 5,6 3 received 1,2,3,4,5,6,7,8	Process 4 Outgoing channels 3 sent 1,2,3 Incoming channels 2 received 1,2 stored 3,4

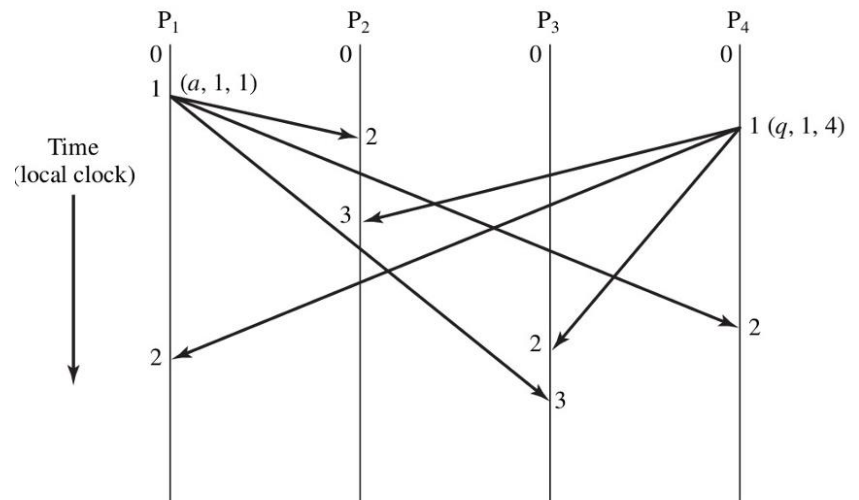
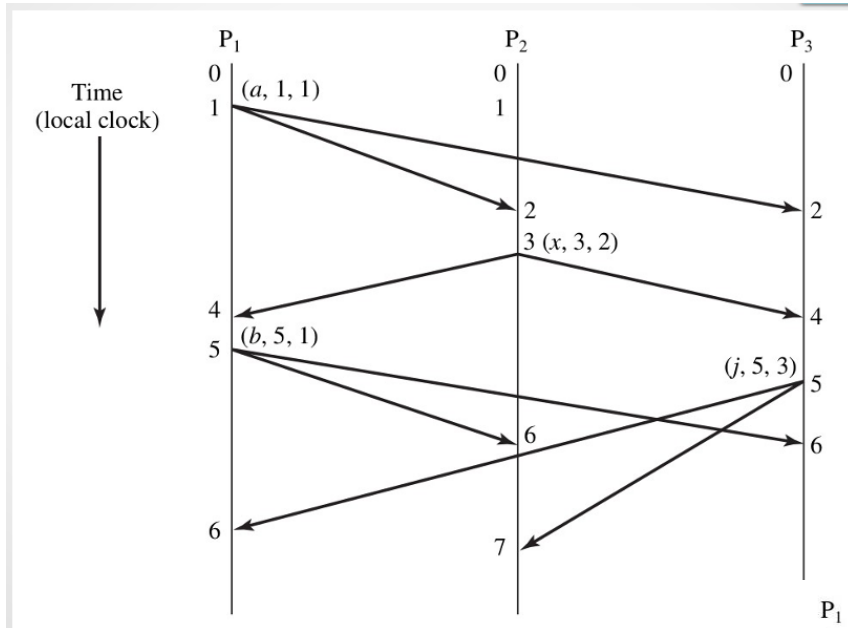
54. Distribuirano međusobno isključenje

Samo jedan proces sme da bude u kritičnoj sekciji. Proces koji se zaustavlja u sekciji koja nije kritična ne sme da spreči ostale da uđu u kritičnu sekciju. Ne sme da čeka beskonačno dugo na ulazak, a u k.s. ostaje konačno dugo.

Distribuirano rešenje: svi čvorovi imaju približno jednak broj informacija, svaki čvor ima samo delimičnu sliku, otkaz čvora ne obara ceo sistem, ne postoji generator takta za vremensku sinhronizaciju.

55. Algoritam vremenskog žiga

Svaki proces ima svoj brojač (generator takta) C_i . Poruka se šalje u obliku (m, T_i, i) – (sadržaj, timestamp, oznaka računara). Kada se poruka primi na prijemni sistem, j postavlja vrednost svog generatora takta na $C_j = 1 + \max(T_i, C_j)$. Za poruku x sa i računara se kaže da prethodi poruci y sa j računara ako $T_i < T_j$ ili $T_i = T_j$ i $i < j$. Na osnovu ovih pravila možemo uvesti redosled poruka što nam kasnije omogućava sinhronizaciju i pravljenje redova za čekanje.



56. Lamportov algoritam distribuiranog reda čekanja

Pretpostavke modela: Distribuirani sistem se sastoji od N čvorova. Svaki čvor sadrži proces koji zahteva uzajamno isključenje u ime svih ostalih procesa.

Poruke se primaju u redosledu u kom su poslate. Sve poruke se isporučuju ispravno, mreža je potpuno povezana, nema posrednika.

Svaki čvor mora da ima kopiju reda čekanja. Neophodno je da proces primi poruke od svih čvorova kako bi bio siguran da nema poslatih poruka koje su još na putu. Svaki čvor ima svoj red čekanja čiji član $q[i]$ sadrži poruku od procesa P_i , a poruke mogu biti:

- (Zahtev, T_i, i) – zahtev da proces koristi neki resurs
- (Odgovor, T_j, j) – dozvoljavanje pristupa resursu kojim upravlja
- (Prepuštanje, T_k, k) – prepušta prethodno dodeljen resurs

Verzija 1:

Kada proces P_i zahteva pristup do nekog resursa, izdaje zahtev (Zahtev, T_i, i). Ovaj se zahtev smešta kao $q[i]$ i šalje poruku ostalim procesima. Ako P_j ne sadrži zahtev, odmah šalje (Odgovor, T_i, j).

P_i može da pristupi resursu ako su ispunjena oba uslova (sopstvena poruka sa zahtevom je najranija poruka u q i sve ostale su nakon nje).

P_i prepušta resurs izdavanjem poruke (Prepuštanje, T_i, i). Poruka se kopira u $q[i]$ i šalje ostalima. Kada P_i primi poruku za prepuštanje kopira je u $q[j]$. Isto i za Odgovor poruku.

Ovim algoritmom se postiže uzajamno isključenje, pravednost (po redosledu vremenskih žigova), nema uzajamnog blokiranja i nema praznog hoda (slanjem poruke prepuštanje).

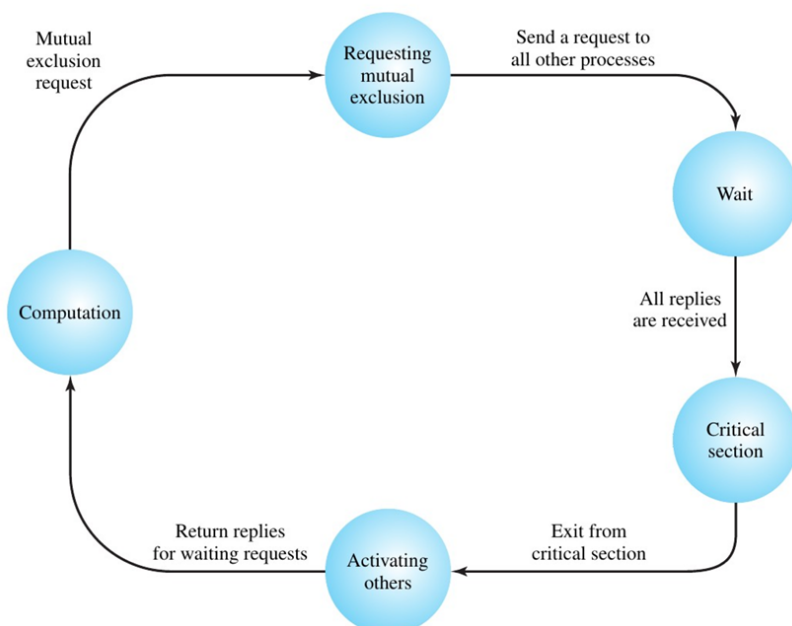
Verzija 2: bez poruke Propuštanje

Kada proces P_i zahteva pristup, izdaje se Zahtev poruka. Smešta se u $q[i]$ i šalje poruka ostalim procesima. Kada proces P_j primi poruku zahtev, on radi po sledećim pravilima:

- ako je P_j u svojoj k.s, odlaže odgovor
- ako P_j ne čeka da uđe u svoju k.s, šalje Odgovor
- Ako P_j čeka da uđe u svoju k.s. i ako pristigla poruka kasni u odnosu na njegov zahtev, smešta poruku u $q[i]$ i odlaže slanje Odgovora
- Ako P_j čeka da uđe u svoju k.s. i ako pristigla poruka prethodi njegovom sopstvenom zahtevu, on poruku smešta u $q[i]$ i šalje Odgovor

P_i može da pristupi resursu kad primi poruke sa Odgovorom od svih ostalih procesa

Kada P_i napusti svoju k.s, on prepušta resurs tako što šalje poruke sa odgovorom svim procesima koje imaju zahteve na čekanju.



57. Uzajamno isključenje pomoću prosleđivanja tokena

Samo proces koji ima token može da uđe u svoju kritičnu sekciju gde je token u stvari niz tokena u čiji je k-ti element upisan vremenski žig sa vremenom kada je token bio poslednji put kod procesa P_k. Svi procesi održavaju niz sa zahtevima u čiji je j-ti element upisan vremenski žig sa vremenom poslednjeg zahteva od procesa P_j.

```
if (!token_present) {
    clock++;
    broadcast (Request, clock, i);
    wait (access, token);
    token_present = true;
}
token_held = true;
<critical section>;

token[i] = clock;
token_held = false;
/* Postlude */
for (int j = i + 1; j < n; j++) {
    if (request(j) > token[j] && token_present) {
        token_present = false;
        send (access, token[j]);
    }
}
```

(a) First part

```
if (received (Request, k, j)) {
    request (j) = max(request(j), k);
    if (token_present && !token_held)
        <text of postlude>;
}
```

(b) Second part

Notation

<code>send (j, access, token)</code>	end message of type access, with token, by process j
<code>broadcast (request, clock, i)</code>	send message from process i of type request, with time-stamp clock, to all other processes
<code>received (request, t, j)</code>	receive message from process j of type request, with time-stamp t

58. Zookeeper - koncept Znode-a (proveri)

Zookeeper rešava ove probleme koristeći svoj sistem datoteka sa magičnom strukturom drveta koji se zove **znodes**, donekle sličan Unix sistemu datoteka. Ovi znodesi su analogni folderima i fajlovima u Unix sistemu datoteka sa nekim dodatnim magičnim sposobnostima.

Zookeeper obezbeđuje primitivne operacije za manipulaciju ovim znodovima, preko kojih ćemo rešavati probleme našeg distribuiranog sistema.

Ključne Znode funkcije koje treba da znate:

- Znodes može da skladišti podatke i da ima decu Znode u isto vreme
- Može da skladišti informacije kao što su trenutna verzija promena podataka u Znode-u, ID transakcije poslednje transakcije izvršene na Znode-u.
- Svaki znode može imati svoju listu kontrole pristupa (ACL), kao što su dozvole u Unik sistemima datoteka. Zookeeper podržava: kreiranje, čitanje, pisanje, brisanje, dozvole administratora (podešavanje/uređivanje dozvola).
- Znodes ACL takođe podržava autentifikaciju zasnovanu na korisničkom imenu/lozinki na pojedinačnim znodesima.
- Klijenti mogu da podese sat na ovim znodovima i dobiju obaveštenja ako dođe do bilo kakvih promena u ovim znodovima.

Ove promene/događaji mogu biti promena u podacima znodes-a, promena u bilo kom od znodes dece, kreiranje novog podređenog Znode-a ili ako je bilo koji podređeni Znode izbrisan ispod znode-a na kojem je postavljen sat.

Primer

Zookeeper je kao velika knjiga sa posebnim listovima gde možete zapisivati razne stvari. Svaka stranica te knjige ima svoje ime i možete na njoj pisati informacije.

Ove stranice mogu čuvati informacije kao što su brojevi ili rečenice, a takođe mogu imati i podstranice. Možete odlučiti ko sme da čita ili piše na određenoj stranici i možete pratiti kada su informacije poslednji put menjane.

Takođe, ako postavite sat na neku stranicu, možete dobiti obaveštenja kada se nešto promeni na toj stranici ili kada se doda nova podstranica.

Ovo je korisno za organizaciju i deljenje informacija u različitim situacijama, kao što su velike kompanije ili distribuirani računarski sistemi. Zookeeper pomaže da se održi red i da se različiti ljudi ili računari usklade i rade zajedno.

59. Zookeeper - algoritam izglasavanja leader-a (1) (proveri)

Jedan od poznatih algoritama za izbor lidera u Zookeeperu je algoritam "**Zabranjivanje**" (eng. "**Bully Algorithm**"). Evo kako on funkcioniše:

Inicijalizacija: Algoritam počinje tako što svaki čvor u distribuiranom sistemu ima neki identifikacioni broj ili rang. Inicijator izbora (obično čvor koji inicira izbor lidera) šalje poruku "IZABERI" sa svojim rangom svim čvorovima sa višim rangom od svog.

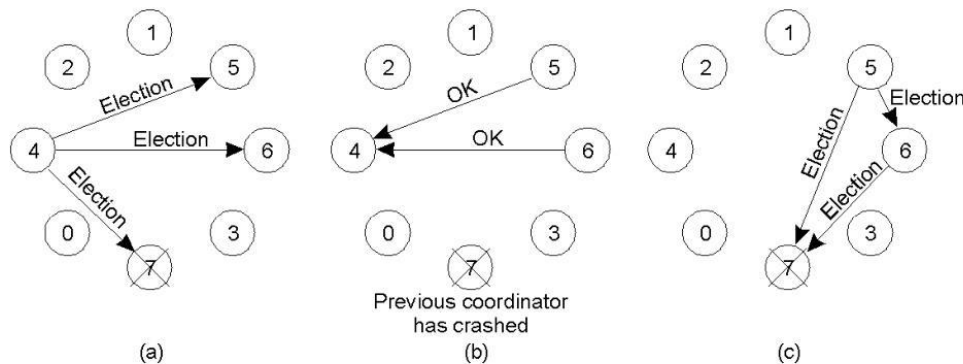
Poruke "IZABERI": Kada čvor prima poruku "IZABERI" od nekog drugog čvora, on proverava da li je njegov rang veći od ranga čvora koji je inicirao izbor. Ako jeste, čvor šalje poruku "ODUSTANI" onom čvoru koji ga je izazvao. Ako njegov rang nije veći, čvor šalje poruku "IZABERI" dalje ka čvorovima sa višim rangom.

Poruka "ODUSTANI": Kada čvor prima poruku "ODUSTANI," znači da postoji neki drugi čvor sa većim rangom koji je preuzeo ulogu lidera. U tom slučaju, čvor prestaje sa slanjem poruka "IZABERI" i obaveštava ostale čvorove da je odustao od kandidature za lidera.

Pobednik: Kada se završi ovaj proces slanja poruka, čvor sa najvišim rangom koji nije dobio "ODUSTANI" poruku postaje lider. On obaveštava sve ostale čvorove da je sada lider i preuzima odgovornosti lidera, kao što je koordinacija rada u sistemu.

Algoritam "Zabranjivanje" osigurava da čvor sa najvišim rangom postane lider, a ostali čvorovi ga priznaju kao lidera. Ovaj algoritam je efikasan i omogućava brz izbor lidera u distribuiranim sistemima. Međutim, važno je napomenuti da Zookeeper koristi složenije algoritme za izbor lidera kako bi se osigurala visoka dostupnost i otpornost na kvarove u distribuiranim okruženjima.

The Bully Algorithm (1)



The bully election algorithm; pcs 7 as coordinator=> crashed

- (a) Process 4 is the first on noticed the crashed >> send ELECTION process to 5, 6 and 7 (higher pcs)
- (b) Process 5 and 6 respond, telling 4 to stop
- (c) Now 5 and 6 each hold an election

31

60. Zookeeper - algoritam izglasavanja leader-a (2) (proveri)

Izbor lidera (leader election) u Zookeeper distribuiranom sistemu se vrši pomoću algoritma poznatog kao "**Zabranjivanje**" (eng. "**Zookeeper Leader Election Algorithm**"). Evo detaljnijeg opisa kako ovaj algoritam funkcioniše u okviru Zookeeper-a:

Inicijalizacija i kreiranje privremenih znodes-ova: Svaki čvor u Zookeeper klasteru (distribuiranom sistemu) ima svoj jedinstveni identifikator i želi da postane lider. Da bi započeli izbor lidera, svi čvorovi privremeno kreiraju poseban "privremeni znode" (engl. "ephemeral znode") pod određenim direktorijumom u Zookeeper hijerarhiji. Ovi privremeni znodes-i će biti korišćeni kao kandidati za lidera.

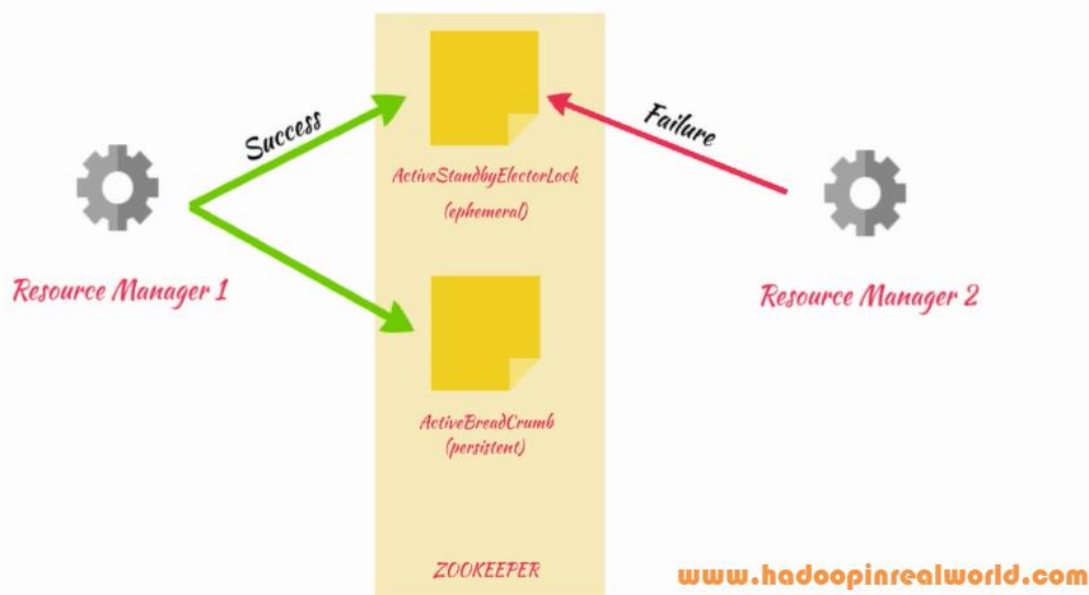
Poređenje redosleda znodes-ova: Zookeeper garantuje redosled kreiranja privremenih znodes-ova. Kada čvor kreira svoj privremeni znode, Zookeeper mu dodeljuje jedinstven redni broj (sekvenca) koji odražava redosled kreiranja. Čvorovi sada porede redosled svojih znodes-ova kako bi utvrdili ko ima najmanji redni broj. Čvor sa najmanjim rednim brojem će postati lider.

Pracenje promena i obaveštenja: Svi čvorovi prate promene u Zookeeper hijerarhiji. Kada neki čvor postane lider (tj. ima najmanji redni broj), ostali čvorovi će to primetiti putem promene u hijerarhiji. Svaki čvor koji nije lider sada zna ko je lider i može komunicirati s liderom za dalju koordinaciju i upravljanje resursima.

Liderova odgovornost: Nakon što postane lider, čvor preuzima odgovornosti lidera, kao što su koordinacija upita iz aplikacija i odlučivanje o tome koji čvorovi mogu obavljati određene zadatke.

Važno je napomenuti da Zookeeper koristi složenije strategije za izbor lidera koje garantuju visoku dostupnost i otpornost na greške. Na primer, Zookeeper koristi kvorume (engl. "quorums") kako bi omogućio izbor lidera i rad u slučaju otkazivanja čvorova. Ovo osigurava da čak i u nepredviđenim situacijama, sistem i dalje može funkcionisati sa pouzdanim liderom.

Leader Election



61. Network File System (NFS)

Primer

NFS je kao da imaš veliki ormar pun knjiga i datoteka kod kuće, ali želiš da te knjige i datoteke budu dostupne i tvojim prijateljima koji žive u drugim kućama. NFS omogućava da tvoji prijatelji dolaze do tvojih knjiga i datoteka preko interneta, kao da su kod kuće.

To radi tako što tvoj računar sa datotekama deli te knjige i datoteke preko mreže, a tvoji prijatelji koriste svoje računare da pristupe i čitaju te datoteke. Tvoj računar postaje poput knjižare u koju tvoji prijatelji mogu ući i pozajmiti knjige koje im trebaju.

NFS omogućava da se datoteke dele i koriste na različitim računarima kao da su svi oni povezani u istoj mreži. To je korisno kada želiš da deliš informacije sa drugima ili da pristupiš svojim datotekama sa različitih uređaja bez potrebe za fizičkim kopiranjem ili prenosom datoteka.

NFS je distribuirani sistem za deljenje datoteka koji omogućava korisnicima da pristupaju datotekama smeštenim na udaljenim serverima preko mreže kao da su lokalne datoteke. Ovo je korisno u okruženjima gde više računara treba da deli i upravlja istim setom datoteka.

Distribuirani fajl sistem protokol.

Omogućava administratorima da dele sve ili deo fajl sistema na serveru u svrhu pristupa udaljenih korisnika.

Koristi RPC za rutiranje zahteva između klijenata i servera.

Prednosti su to što je open, jeftin, centralizovan, distribuiran i siguran.

Mane- zavisi od rpc (neophodno je da bude korišćen na sigurnoj mreži) i ograničen je brzinom protoka podataka i skalabilnošću. mount komanda se koristi za pristup podacima sa servera.

